Centre number 50723 Candidate number 4373 Luke Say

3D GRAPHICS RENDERING WITH OPENGL

1 Analysis 5
1.1 Introduction 5
1.2 Background 6
1.3 Current Solutions 7
1.3.1 Unity 7
1.3.2 OGRE 8
1.4 User Requirements 9
1.4.1 User survey 9
1.4.2 Client interview 12
1.5 Objectives 13
1.5.1 Initial objectives 13
1.5.2 Final objectives 14
1.5.3 Extension objectives 15
1.6 Acceptable Limits 16
1.7 Programming Languages 17
2 Documented Design 18
2.1 Program overview 18
2.2 Ownership verification 20
2.2.1 License checking 20
2.2.2 License creator 24
2.3 File loading 27
2.3.1 Overview 27
2.3.2 Image loading 28
2.3.3 Mesh loading 30
2.3.4 Shader loading 31
2.4 Matrices 37

2.5 Shader code 44

2.6 Rendering classes 51

3 Technical Solution 58

3.1 main.cpp 58

3.2 fileHandler 62

3.2.1 fileHandler.h 62

3.2.2 fileHandler.cpp 66

3.3 licenseChecker 82

3.3.1 licenseChecker.h 82

3.3.2 licenseChecker.cpp 83

3.3.3 server.py 86

3.3.4 keyCreator.py 90

3.4 hashtable 93

3.4.1 hashtable.h 93

3.4.2 hashtable.cpp 96

3.5 graphicMath 97

3.5.1 graphicMath.h 97

3.5.2 graphicMath.cpp 102

3.6 graphicHandler 107

3.6.1 graphicHandler.h 107

3.6.2 graphicHandler.cpp 113

3.7 shaders 125

3.7.1 vertexShader.vert 125

3.7.2 fragmentShader.frag 127

4 Testing 130

4.1 Features testing 130

4.2 Functionality testing 130

5 Evaluation 131

5.1 Completeness 131

5.2 Improvements 132

1 Analysis

1.1 Introduction

3D graphics has always been a highly researched topic, as it allows for tangible visual output to the user, paving the way for CAD software, physics simulators and video games. My investigation aims to create an application that allows developers to render 3D geometry through files that specify an objects vertices while having the ability to build more functionality on top of this.

1.2 Background

Many applications use 3D graphics, and some of the companies may choose to build their own framework, with others using a pre-existing framework. Typically, a graphics API is used as this means the GPU is utilised, as hardware manufacturers specifically design their device drivers to support the three main graphics API. These are Vulkan, OpenGL and DirectX. They all work in a similar way. The vertices specifying the object to be rendered will go through stages in the graphics pipeline, with some of these stages being programmable using shader languages. These give graphics developers access to the vertex, tessellation, geometry and fragment shaders while abstracting away the parts graphics developers wouldn’t need to use. The shader language for Vulkan and OpenGL is GLSL, while DirectX uses HLSL. My investigation will use OpenGL with GLSL, as DirectX and HLSL are owned by Microsoft, so it only works on Windows platforms, and I believe Vulkan is too verbose for the investigations needs.

1.3 Current Solutions

1.3.1 Unity

unity.com

Unity is the one of the most popular game development engine, with almost 60% of all games released being built using unity, as of 2018. However, although unity is marketed as a game engine, developers can use it for other applications as well.

Unity has built in 3D graphics, supporting custom shaders written in either GLSL or HLSL. It also has many required features already built into the engine, so developers don't have to spend their time creating and optimising basic infrastructure for their applications

Unity is both free and intuitive to use as it has a well designed GUI. This attracts a lot of hobby game developers as it means they don't have to spend much time researching how to use this platform and they don't have to spend money to just develop games. This is part of why unity is so popular.

Advantages

- Free for hobby developers
- Easy to use
- Built in physics engine and camera
- Custom shader support
- 2D and 3D support

Disadvantages

- Unity watermark on compiled games
- Paid for games that make a certain revenue
- Proprietary

1.3.2 OGRE

ogre3d.org

OGRE is an open-source graphics engine that gives developers the option to build custom 3D applications without boxing themselves in to a real-time game engine. It acts as an API that developers can use to avoid writing their own OpenGL application.

OGRE is designed to allow developers to use their own shaders, however it still has default shaders if the developer doesn't need to write custom shaders. It supports both HLSL and GLSL, however it also has its own shading language, Cg, which serves as a system agnostic shading language for either DirectX or OpenGL / Vulkan systems. This makes it very cross platform, working for Mac, Windows and Linux systems.

OGRE is an API, so it doesn't have a GUI to see features with, forcing developers to read through many pages of documentation to learn how to use it. However, this makes OGRE a very powerful tool, with developers being able to use it for a lot more solutions than a traditional game engine.

However, OGRE does use the assimp library for its texture loading which has been known to have security issues.

Advantages

- Open-source
- Versatile
- Agnostic shader support
- Free for anyone
- Lightweight API

Disadvantages

- Not intuitive
- No 2D support
- Potential security issues

1.4 User Requirements

To ensure that my project is up to user standards, I have decided to do some preliminary research before planning my project.

1.4.1 User survey

I first released a survey to determine what a general user base found important in graphics rendering engines.

Q1: What is the most important aspect in graphics engine

The amount of frames per second (speed of the program)

6 votes

How good the graphics look

4 votes

How many meshes can be loaded at once

0 votes

The data suggests that both performance and quality of graphics are equally as important to users. I can infer that I should strive to have a preformative program and have good looking graphics

Q2: What would you say is the largest factor to making graphics look good

Detailed models

5 votes

Lighting

3 votes

Realism

1 vote

While I cannot create detailed models (as that is a 3D modelling problem and not a programming problem), I can create lighting. Lighting has a similar amount of votes to detailed models so it suggests that it is of similar importance to the quality of models.

Q3: How would you prefer to verify your ownership of a program

License key

7 votes

Scanning a QR code on an app

2 votes

Other

1 vote

The results here clearly suggest license checking should be the method to verify user ownership of the program.

Q4: If you are a developer, what would make a graphics engine desirable to code with

Easy to understand (by abstracting away OpenGL API elements)

2 votes

Well documented

2 votes

Other

0 votes

Not every user had to vote on this question, and it resulted in a tie. However, achieving both wants should be easy to do in the time frame of this project by commenting my code well and by introducing classes to simplify creating meshes.

1.4.2 Client interview

After this, I scheduled an interview with a developer and to gain further insight into what a developer thinks a graphics rendering engine should have.

Q. How do users typically like to move their camera around?

Generally the W, A, S and D keys or the arrow keys

Q. What is your opinion on Linux support?

I think it's safe to assume that the overwhelming amount of people use Windows instead of Linux and/or Mac and those systems already have pretty accurate Windows emulators

Q. What file formats are typically used by 3D modellers and graphic designers?

.FBX .MA .ZTL .STL .PNG .JPG .Webp and .OBJ. If these aren't in the program the developers themselves should have the ability to be able to easily implement their own formats whenever needed

Q. What's your opinion on the industry standard graphics API's?

Out of the big three (DirectX, Vulkan and OpenGL) OpenGL stands out to be simpler while still being highly accessible whereas DirectX has the least support cross platform and it's much more complicated, Vulkan similarly but with the most support

1.5 Objectives

1.5.1 Initial Objectives

These are a list of loose objectives I will follow during the course of my project:

- A window to display a scene
- Scenes can contain any amount of meshes
- Performance of the application is good enough that movement seems smooth
- Meshes can be given various colours and textures
- Meshes can be any shape / model
- Users can interact and change scenes
- The application can verify that a user ‘owns’ it
- Debugging features

1.5.2 Final Objectives

These final objectives are more specific project and in-depth requirements:

Application

1. The ability to create meshes and render these meshes from files that specify the all the vertices in the model
2. The ability to control the position, orientation and size of the mesh programmatically
3. The ability to move and rotate the entire rendered scene in real time to emulate a camera
4. The ability for meshes to use and switch between various shader programs
5. The ability to colour and texture meshes with any image or colour.
6. Lighting model

User ownership verification

1. Client-server model for user license verification
2. Database to store licenses and any associated data
3. The ability to generate random, unique licenses

1.5.3 Extension Objectives

I have also thought of a list of objectives that would increase the solution effectiveness, but are not necessary for the projects success.

1. Physics simulation
2. Syncing the graphics engines FPS to the monitors refresh rate
3. Blending edge pixels with background pixels to make mesh edges appear smoother

1.6 Acceptable Limits

These features would normally be required in a genuine product, however, for this project, I have deemed these objectives fall outside of the scope of it for various reasons.

- Support for multiple image and mesh file types. Normally 3D modellers and artists work with many different file types when creating their meshes and textures, which all require different methods of extracting the data. This would be a huge waste of time as once the data is extracted from the file, it is used in the exact same way.
- A way to obtain licenses. Customers should have a way to buy licenses otherwise the product is worthless. This would require creating a website then implementing a payment gateway, which typically costs money and would also prove difficult to debug.
- Server verification daemon being hosted on a cloud server. For users to be able to verify, the server-side verification daemon would need to be hosted on a server accessible on the internet. Self hosting would require me to open home network ports to the internet, which could prove to be risky and hosting on a cloud server costs money.

1.7 Programming Languages

I believe for the client-side, C++ will be the best language to use. This is because C++ is a compiled language and is designed to create fast programs over everything else, making it a good choice for this project as being able to render frames quickly is imperative. The OpenGL API also natively supports C++, meaning I need to use less libraries, making the client-side program take up less storage, improving the user experience.

For the server-side, I've decided to use python. Python is a language I'm already quite familiar with. Python also abstracts many elements, this makes creating python applications much quicker than in another language. The speed of the server-side program is also not critical, neither is the security. This makes python attractive for this use case.

2 Documented Design

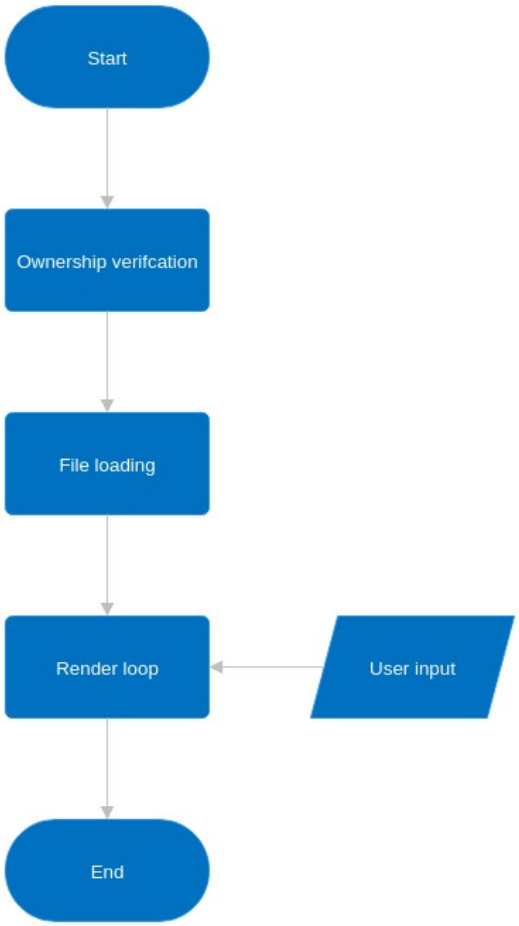
2.1 Program overview

This project will have 3 critical areas

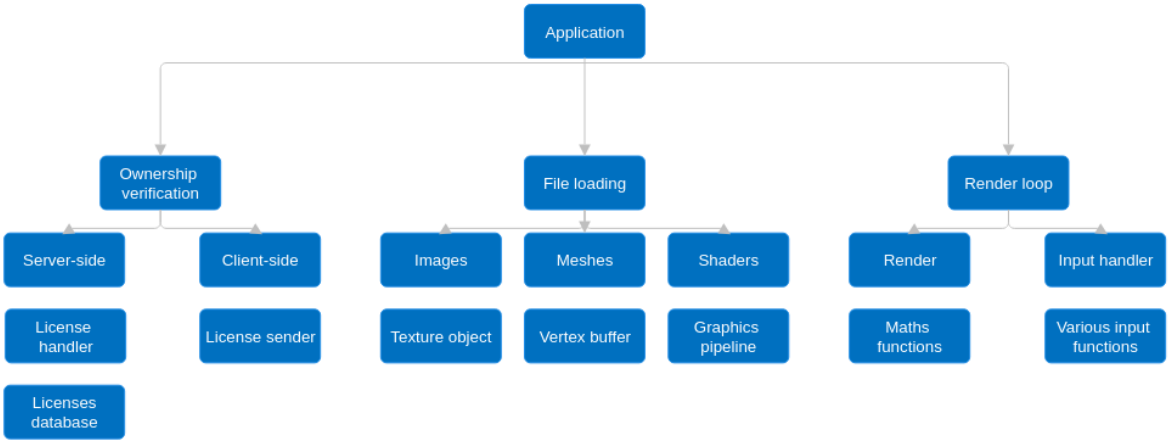
These are:

- Ownership verification
- File loading
- Rendering loop

The application goes through these critical areas during its lifetime according to this flowchart



These critical areas can all be broken down into various different functions. To display this I have created this hierarchy chart.



2.2 Ownership verification

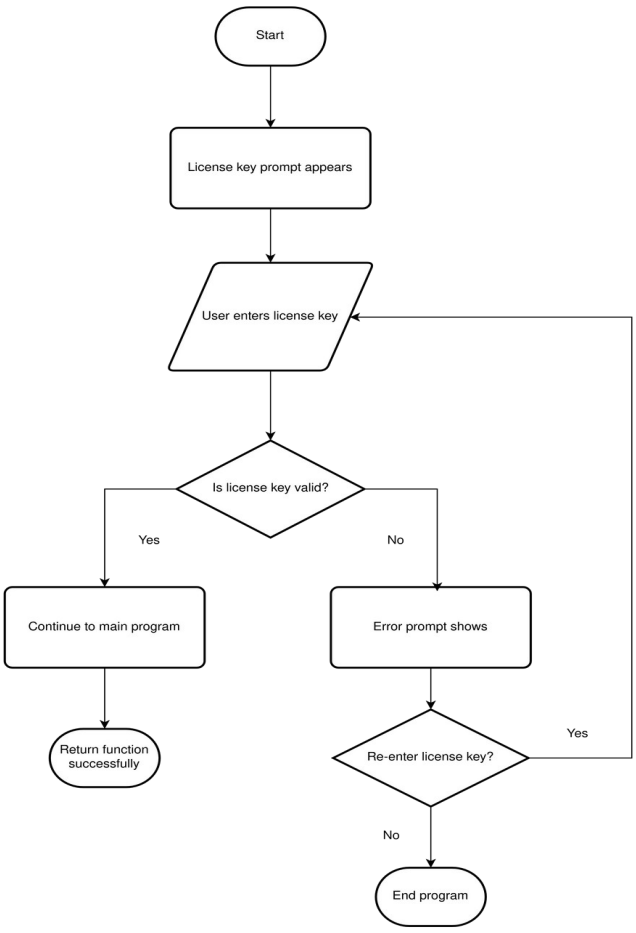
As I've stated in the objectives, users should be able to input a license key to verify their ownership of the application. However, so I can identify if a license key has already been used and who used it, I've decided that users should also be sending an email along with the license key. To make this simple, the client will format the data into a single json encoded string, which will be sent to the server. This means the server can still easily extract sent data and can also use the HTTP protocol without much trouble.

So it's easy to keep track of valid license and emails, this will all be stored in a database on the server. When a client sends data, the server can query the database then send back a response code depending on whether the license and email was valid or not. The server will be checking for data that has been sent to http://127.0.0.1/licenses

2.2.1 License checking

<u>Client-side</u>

To implement these features, I first designed a flowchart for the basic behaviour of this feature.



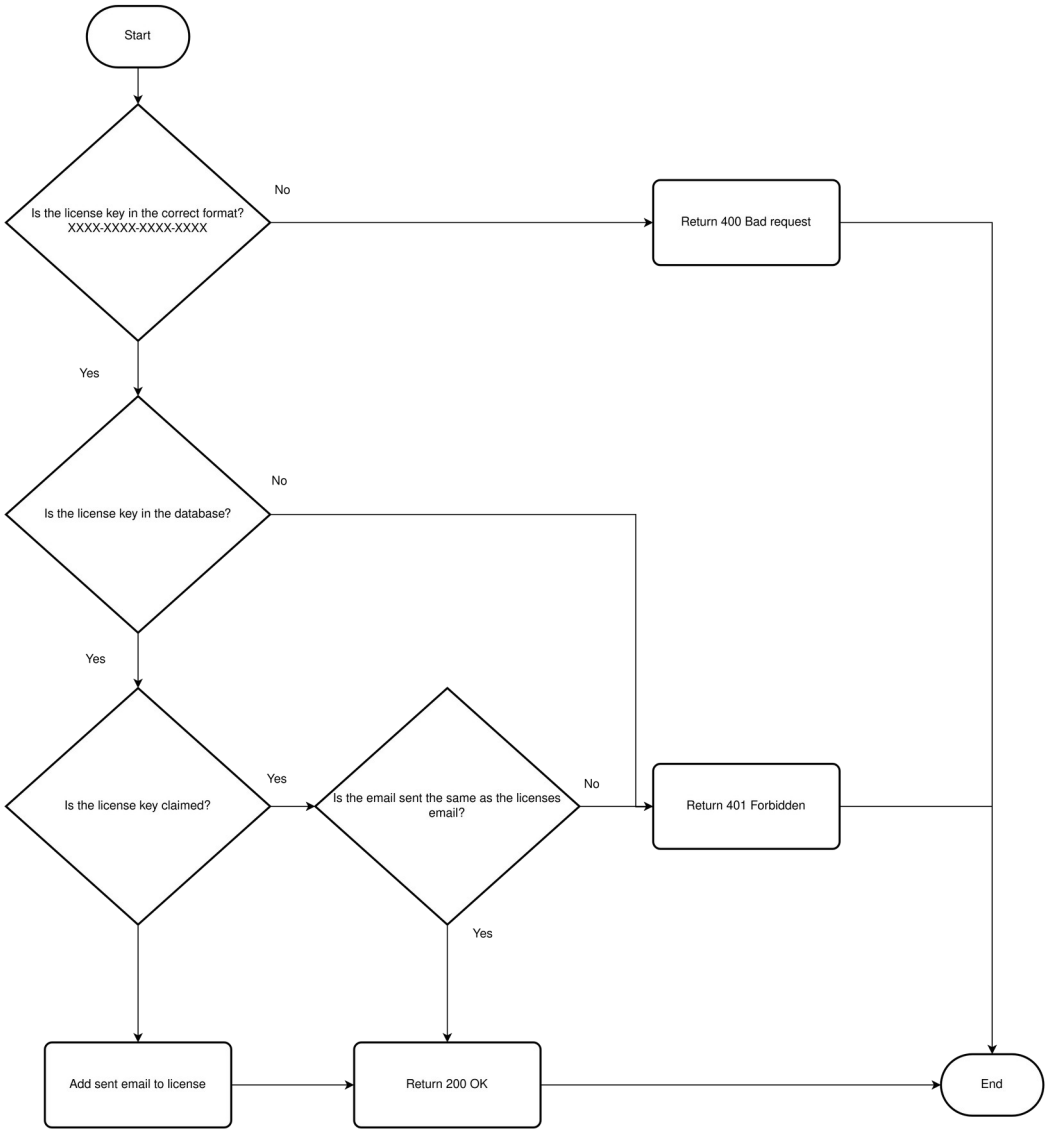
The application first shows a prompt that users will enter their license key and email into. This prompt should never appear again once the user enters a valid license key and email.

If a user enters an invalid license key and email combination, the error prompt will show where they will decide to either re-enter their license key and email or to quit the application.

Server-side

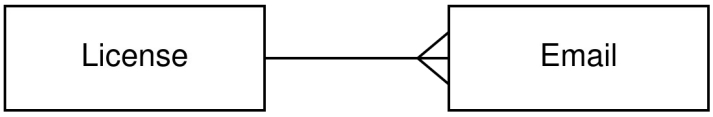
As I've already mentioned, the server will host a simple HTTP application. Clients will use HTTP POST requests to send data to it on http://127.0.0.1/licenses

Flowchart representing the server-side license checking process



The server will use a simple database that stores licenses and emails, with licenses being the primary key of this database.

Entity relationship diagram between licenses and emails



The email column is allowed to have empty fields. This will indicate that a license is unclaimed. Records containing empty email fields will be updated to the email a client provides when they send a license key matching that record. The email field should not be changed if the field is not empty.

Client license keys and emails will only be considered valid if:

1. The license key they provide is stored in the database
2. The record containing the license key has an empty email field or the email they provide matches the email already stored in the record

Once the server has come to a conclusion on the validity of a license key and email a client has provided, it will reply with a HTTP response code.

These are:

- Valid: 200
- Invalid: 401
- Erroneous data: 400

Although this feature isn't used by the client, it is still a good idea to make sure the server can recognise and ignore erroneous data.

Clients will then be able to determine what their next course of action will be according to these response codes.

2.2.2 License creator

Since the server only uses a very simple database, I will also add being able to view and reset the database from the same program in addition to the license generation.

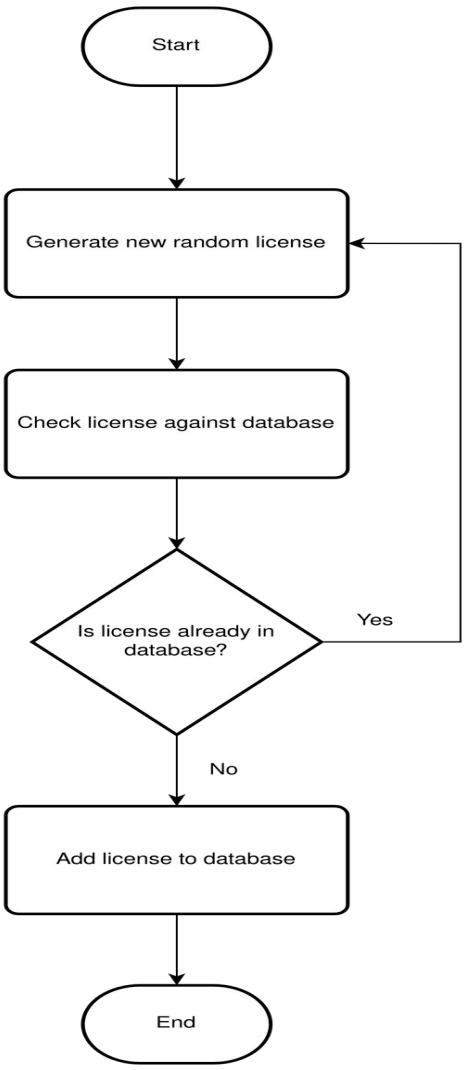
This program will only ever be used by developers, so there will not be a GUI added to it.

<u>License generation</u>

Random licenses need to be able to be created. Licenses will take the form

XXXX-XXXX-XXXX-XXXX where the X's are any letter between A and Z.

The behaviour of this feature should look like this:



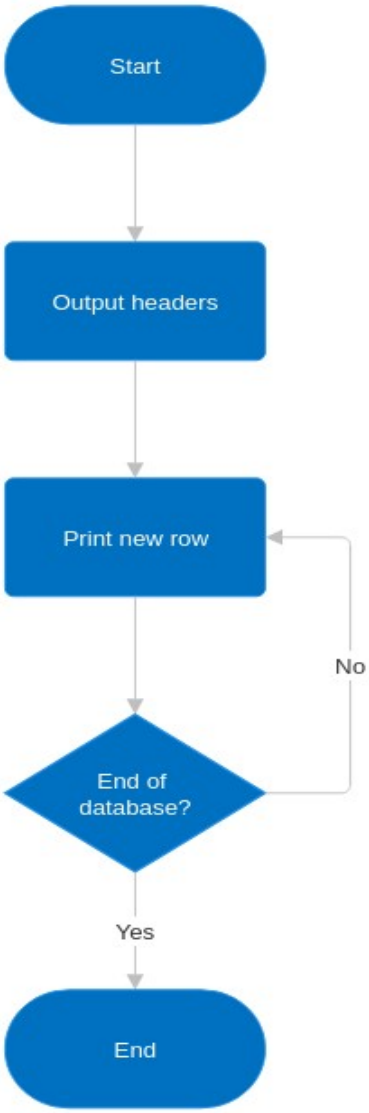
Because the license keys are the primary key of this database, I thought it was necessary to ensure that the new, randomly generated license key is not a duplicate by adding a check.

Database viewer

The database viewer will output the entire database in this format

Licenses	Emails
ABCD-EFGH-IJKL-MNOP	user@example.com
...	...

This will be accomplished by following this flowchart:



Database eraser

This procedure will just delete the databases table, then recreate a new table with the same columns, effectively erasing it.

```
PROCEDURE tblErase
    DROP licenseTbl
    CREATE TABLE licenseTbl(license CHAR(19), email VARCHAR(100))
END PROCEDURE
```

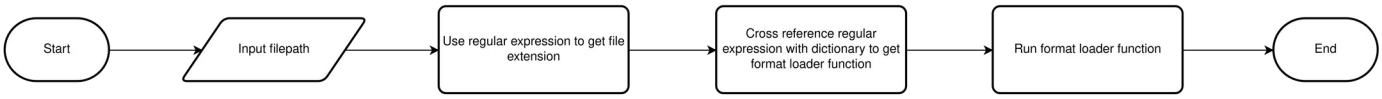
2.3 File loading

2.3.1 Overview

There are a few file types that must be loaded during the runtime of the program (images, meshes, etc.). These all have various different file formats that need to be loaded in different ways.

To streamline this, each file type will have a wrapper function that detects the file format, then uses the correct loading function.

It's done like this over having one wrapper function that detects the file type and then the file format as C++ is typed language, and I would end up having to return memory locations, then cast them to the correct type – which would be much more complex than managing a few different functions.



As mentioned earlier, I have decided to only implement one file format for each file type. However, I will not neglect being able to add new file formats with ease.

This will be accomplished by using a dictionary, using the file extension as the key and a function pointer that loads the associated file format as the stored data.

An example dictionary to demonstrate this:

```
DICTIONARY formatloadersdictionary = {
        "format1": format1-loader-pointer,
        "format2": format2-loader-pointer,
        "format3": format3-loader-pointer
}
```

Format loading functions can be fetched with formatloaders[“format”], meaning once we have a file format, nothing else has to be done with the string to get this function.

All these dictionaries will be stored in a struct, defined as such:

```
STRUCT formatloaders
    DICTIONARY imageformats
    DICTIONARY meshformats
    DICTIONARY shaderformats
    // any other file types that may be needed in the future
ENDSTRUCT
```

There will then be a file loader class, which stores the wrapper functions. It uses an object of the previous class to instantiate it. This is useful as it means I can test different format loading functions without having to touch already functioning code.

fileLoader
-formats: formatloaders
fileloader(formatfunctions: formatloaders) +imageloader(filepath: str) +meshloader(filepath: str) // The rest of the wrapper functions would go here

2.3.2 Image Loading

In order to place images on a rendered mesh, OpenGL requires the image data and its resolution to create a texture.

Since this process of creating the texture should be the same for every image format, it will be in the wrapper function.

All specific format loaders will return a struct, image, that the wrapper function will use to do this.

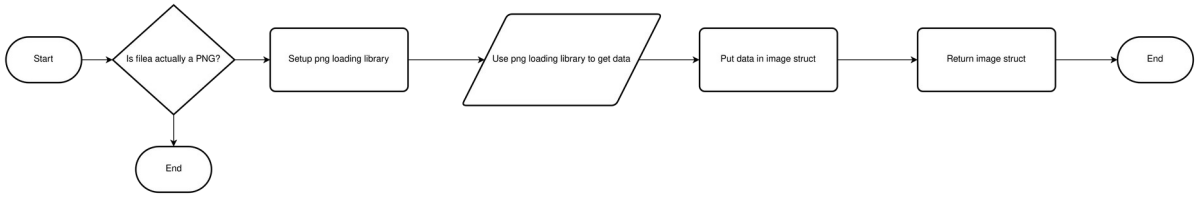
This is what the image struct will look like.

```
STRUCT image
    unsigned int width
    unsigned int height
    unsigned char imagedata
ENDSTRUCT
```

When this struct is passed back to the wrapper function, the members will be passed to the OpenGL API in order to create the texture.

<u>PNG Loading</u>

The only supported image format for this project is the PNG format. Since reading binary data, using decompression algorithms to obtain the image data, then storing it into a memory location falls out of the scope of this project, I will be using a PNG loading library.



2.3.3 Mesh Loading

The OpenGL API stores a meshes vertices inside a vertex buffer. These are then accessed by shaders, which tell OpenGL how to render the vertices stored in the buffer. However, vertices must first be sent to OpenGL. Although these could be stored as a variable in the program code, this is inefficient, hard to visualise and hard to add new meshes to. Also, mesh modelling software does not support this.

All mesh format loaders will construct the meshes vertices from a given mesh file. This will be returned as an array of floats that will be sent to OpenGL to create a vertex buffer. OpenGL also requires a way to determine what floats mean. This information is stored in a vertex array, and the mesh loader will take a function pointer to create the correct vertex array.

The associated vertex buffer and vertex array will be returned in the struct mesh.

```
STRUCT mesh
    unsigned int vertexbuffer      // opengl ids for the vertex buffer and
    unsigned int vertexarray       // array
ENDSTRUCT
```

I have decided to use the OBJ format in order to store mesh vertices as files instead. Mesh modelling software support this file format, and storing as a file means new meshes can just be placed into the correct location and then be used immediately.

OBJ files are stored as plaintext, so I will be making my own reader.

OBJ Reader

```
# Blender 5.0.0
# www.blender.org
mtllib bettercube.mtl
o Cube
v -1.000000 -1.000000 1.000000
v -1.000000 1.000000 1.000000
v -1.000000 -1.000000 -1.000000
v -1.000000 1.000000 -1.000000
v 1.000000 -1.000000 1.000000
v 1.000000 1.000000 1.000000
v 1.000000 -1.000000 -1.000000
v 1.000000 1.000000 -1.000000
vn -1.0000 -0.0000 -0.0000
vn -0.0000 -0.0000 -1.0000
vn 1.0000 -0.0000 -0.0000
vn -0.0000 -0.0000 1.0000
vn -0.0000 -1.0000 -0.0000
vn -0.0000 1.0000 -0.0000
vt 0.998186 0.001814
vt 0.001814 0.998186
vt 0.001814 0.001814
vt 0.999671 0.002282
vt 0.000329 1.001624
vt 0.000329 0.002282
vt 0.001879 1.000074
vt 0.998121 0.003833
vt 0.998121 1.000074
vt 0.998746 0.001254
vt 0.001254 0.998746
vt 0.001254 0.001254
vt 0.996341 0.005612
vt 0.003659 0.998294
vt 0.003659 0.005612
vt 0.995196 0.000898
vt -0.003008 0.999102
vt -0.003008 0.000898
vt 0.998186 0.998186
vt 0.999671 1.001624
vt 0.001879 0.003833
vt 0.998746 0.998746
vt 0.996341 0.998294
vt 0.995196 0.999102
s 0
f 2/1/1 3/2/1 1/3/1
f 4/4/2 7/5/2 3/6/2
f 8/7/3 5/8/3 7/9/3
f 6/10/4 1/11/4 5/12/4
f 7/13/5 1/14/5 3/15/5
f 4/16/6 6/17/6 8/18/6
f 2/1/1 4/19/1 3/2/1
f 4/4/2 8/20/2 7/5/2
f 8/7/3 6/21/3 5/8/3
f 6/10/4 2/22/4 1/11/4
f 7/13/5 5/23/5 1/14/5
f 4/16/6 2/24/6 6/17/6
```

unused

position vertices

normal vertices

texture vertices

unused

face indices

Example of a cube in the OBJ format.

The reader will iterate through every line in the OBJ file.

There will be a series of regular expressions used to extract the numbers in position, normal and texture coordinates – indicated by the starting letter of either v, vn or vt.

These coordinates then be stored in a 2D array for their respective vertex type.

This is how that would look.

```
posvertices = [
    [-1, -1, 1],
    [-1, 1, 1],
    [-1, -1, -1],
    ...
]
```

```
texvertices = [
    [1, 0],
    [0, 1],
    [1, 0],
    ...
]
```

texture coordinates rounded up for simplicity

```
norvertices = [
    [-1, 0, -1],
    [0, 0, -1],
    [1, 0, 0]
    ...
]
```

The face indices (designated with the f) will then be used to construct the vertices. Another regular expression will be used to extract the numbers from each line. This will be stored in a single dimension array as such.

indices = [2, 1, 1, 3, 2, 1, 1, 3, 1, ...]

These indices (organised in [v, vn, vt, ...]) will then be used to access the vertices in each array to construct an array of floats, which is an array of all the relevant vertices for the mesh.

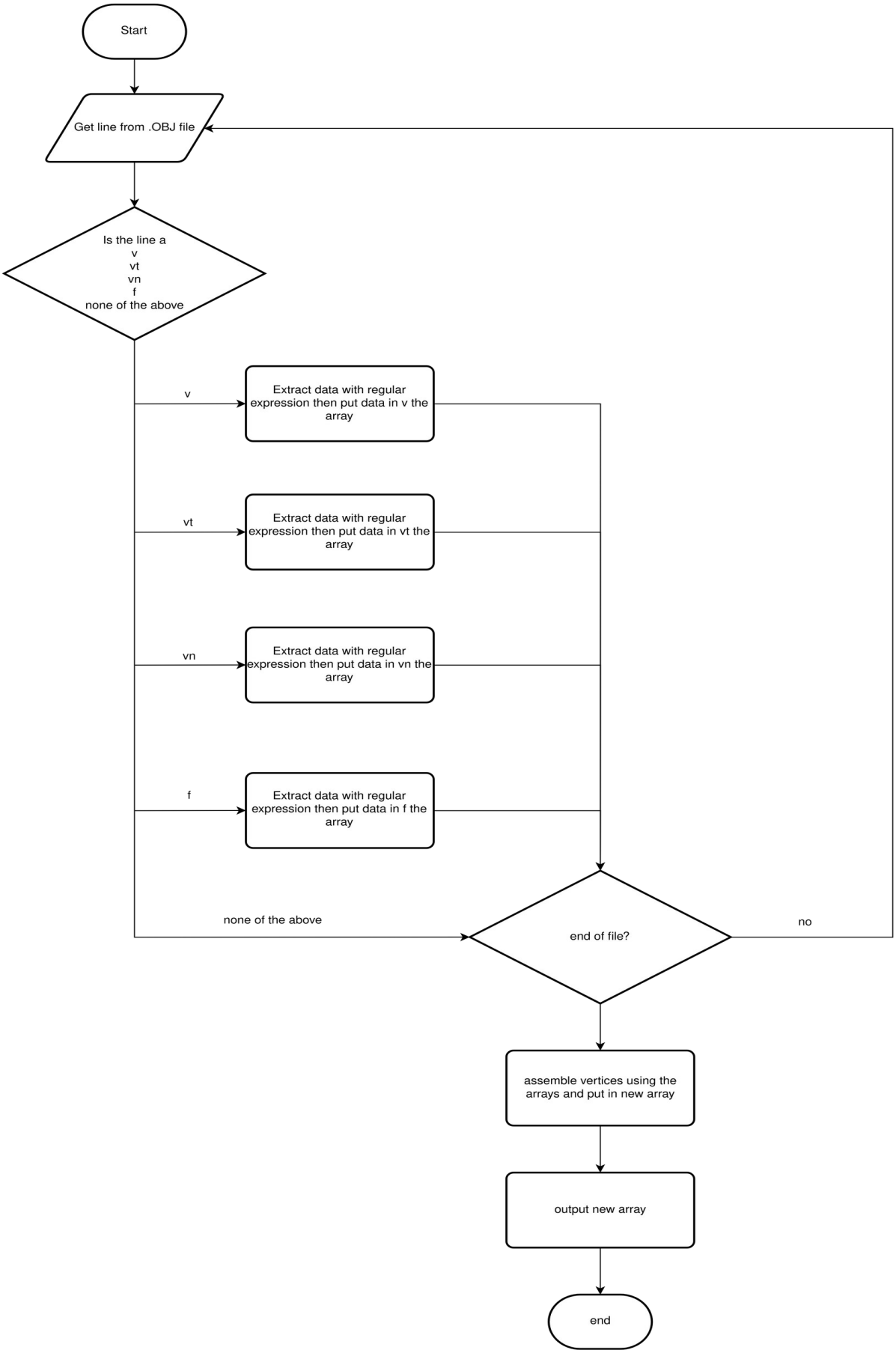
An example of this array, using the first face ‘ 2/1/1 ’

vertices = [-1, 1, 1, -1, 0, -1, 1, 0, ...]

v vn vt

2nd array in posverticies 1st array in norverticies 1st array in texvertices

This is how the reader should work:


Start
Get line from .OBJ file
Is the line a
v
vt
vn
f
none of the above
v
Extract data with regular expression then put data in v the array
vt
Extract data with regular expression then put data in vt the array
vn
Extract data with regular expression then put data in vn the array
f
Extract data with regular expression then put data in f the array
none of the above
end of file?
no
assemble vertices using the arrays and put in new array
output new array
end

2.3.4 Shader Loading

Since the OpenGL API only supports one shader language, GLSL, this will just be the raw format loader.

GLSL files are also plaintext, and OpenGL handles the compilation of shaders, it just requires a correctly formatted string which contains the shader code.

However, a full shader program requires multiple GLSL files. They require a vertex and can optionally have a fragment, tessellation and a geometry shader as well.

The shader compiler will have to take these filepaths as an arguments. Rather than determining what type a shader is from the position of the argument

e.g., function({vertexshader-filepath, fragmentshader-filepath, tessellationshader-filepath, geometryshader-filepath })

the function will take an array of these filepaths

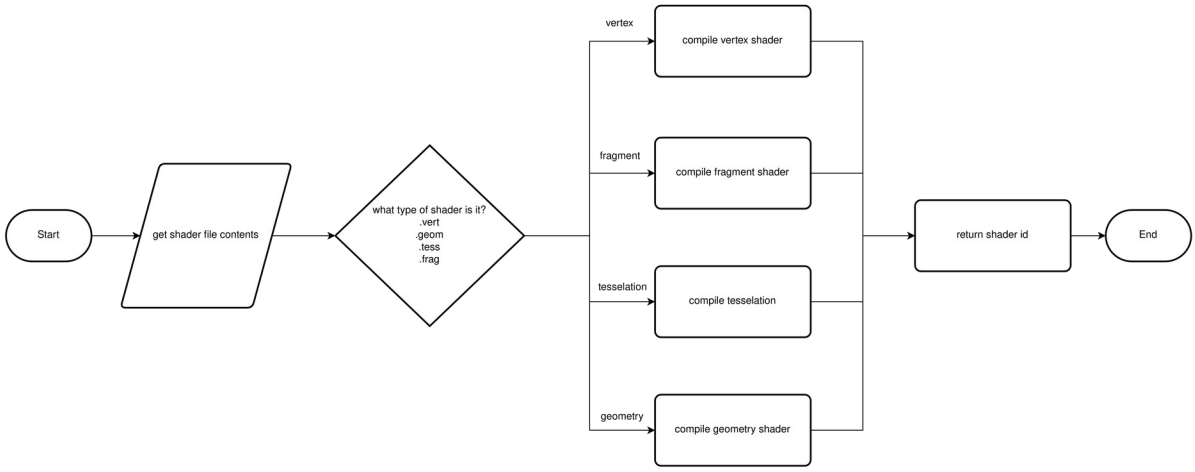
The shader type will now be determined by its file extension instead.

Vertex shader:	.vert
Fragment shader:	.frag
Tessellation shader:	.tess
Geometry shader:	.geom

This method is also better as the shader type position does not have to be documented or remembered, and it is much easier to remember these file extensions.

The shaders OpenGL ID will then be returned in the struct shader – allowing for potential expansion of the shader functionality in the future.

```
STRUCT shader
        unsigned int shaderID
ENDSTRUCT
```



2.4 Matrices

Matrices are used very frequently in this program because they let us transform our vectors through 3D space (e.g., moving a coordinate up by 3 units). Since vectors are just matrices of width 1, they can be handled in the same way.

Matrices will be handled with the class matrix.

Matrix <height: int, width: int>
#array: float[] #identity: bool +height: int +width: int
Matrix(list: float[]) +multiply(othermatrix: Matrix) +multiply(scalar: float) +add(othermatrix: Matrix) +transpose(othermatrix: Matrix) +inverse(othermatrix: Matrix) +getArray()

When a matrix is instantiated, its constructor takes an array of floats, which will be the values inside the matrix, and it takes template arguments height and width. These values will then be copied to the associated members with width and height being constant variables.

The multiply function multiplies two matrices together and returns the result of the multiplication as a new matrix.

This is where the identity boolean will be used, as we can skip the entire matrix multiplication algorithm as when multiplying with or against an identity matrix the other matrix will be returned.

Once the multiplication is complete, we should check if the matrix is an identity matrix (if the matrix is a square matrix) before returning it.

```
PROCEDURE multiply(othermatrix)
        IF self.width != othermatrix.height THEN
                return //error case where programmer is attempting to multiply incompatible matrices
        ENDIF

        matrix newMatrix<self.height, othermatrix.width> // define a new matrix
        IF identity == true THEN //skip case if multiplying against an identity matrix
                newMatrix ← othermatrix //copy othermatrix to newMatrix
                return newMatrix
        ENDIF
        IF othermatrix.identity == true THEN //skip case if multiplying with an identity matrix
                newMatrix ← self  //copy self to newMatrix
                return newMatrix
        ENDIF

        FOR i → self.height DO //the matrix multiplication algorithm
                FOR j → self.width DO
                        FOR k → othermatrix.width DO
                                newMatrix.array[k + i * othermatrix.width] += self.array[j + i * self.width] * othermatrix.array[k + j * othermatrix.width]
                        NEXT k
                NEXT j
        NEXT i

        IF newMatrix.height == newMatrix.width THEN //check if square matrix before checking if identity matrix
                FOR i → newMatrix.height DO
                        IF newMatrix.array[i + newMatrix.height * I] != 1THEN
                                newMatrix.identity = false
                                return newMatrix
                        ENDIF
                NEXT I
                newMatrix.identity = true
                return newMatrix
        ENDIF

        return newMatrix
ENDPROCEDURE
```

Sometimes we need to multiply a matrix by a scalar. The multiply function will have an overloaded function that allows for this.

```
PROCEDURE multiply(scalar)
        matrix newMatrix<height, width>
        FOR i → width * height DO
                newMatrix.array[i] = scalar * self.array[i]
        NEXT i
        return newMatrix
ENDPROCEDURE
```

Matrix addition is simple

```
PROCEDURE add(othermatrix)
        IF width != othermatrix.width AND height != othermatrix.height THEN
                return      //error case if trying to add incompatible matrices
        ENDIF

        matrix newMatrix<width, height>
        FOR i → width * height DO        //iterate through matrices and add values together
                newMatrix ← array[i] + othermatrix.array[i]
        NEXT i

        return newMatrix
ENDPROCEDURE
```

Sometimes we need to transpose matrices as libraries may store their matrices in column-major format rather than row-major (how I've decided to store my matrices) or for some equations involving matrices

```
PROCEDURE transpose()
        matrix<width, height> newMatrix          //reverse width and height for transposition

        FOR i → self.height DO
            FOR j → self.width DO
                newMatrix.array[i + j * self.height] = self.array[j + i * self.width]     //flip elements in arrays
            NEXT j
        NEXT i

        return newMatrix
ENDPROCEDURE
```

Matrices also need to be inverted sometimes. The definition of an inverse matrix is

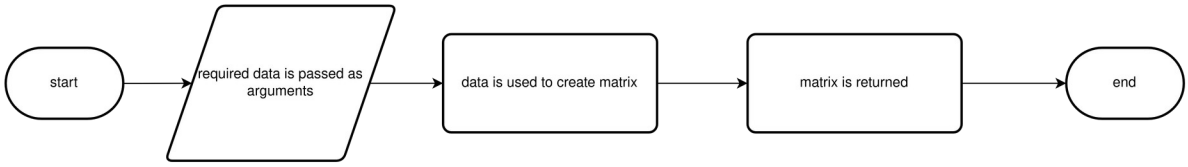
$$M M^{-1} = I$$

where M is any square matrix, M^{-1} is the inverse of M and I is the identity matrix.

```
PROCEDURE inverse()
        IF self.width != self.height THEN
                return      //cant get the inverse of a non square matrix
        ENDIF
        IF self.width > 4 OR self.width < 2 THEN
                return      //matrices above size 4 or below size 2 are not handled
        ENDIF

        matrix newMatrix<self.width, self.width>

        //square matrices of different sizes all have different methods to finding their inverse
        IF self.width == 2 THEN

        ELSE IF self.width == 3 THEN

        ELSE        //assume 4x4 matrix

        ENDIF

        return newMatrix
ENDPROCEDURE
```

In addition to handling matrices, there is also the need to have matrix transformations. These are used very often, so I have decided that there should also be functions to create these. These matrix transformations are all represented as 4x4 matrices because 3x3 matrix transformations cannot change the origin of a 3x1 vector, while 4x4 matrices can change the origin of a 3x1 vector (if there is an extra homogeneous coordinate to allow for multiplication between the vector and the transformation matrix)

These functions will all follow a very simple flow:



I have decided not to include any pseudocode for the transformations, as there is no 'smart' way to create them.

Each element in the matrices array must be set to the correct value

Enlargements

$$T=\begin{pmatrix} k & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & k & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where k is how much the position vector should be enlarged by

Translations

$$T=\begin{pmatrix} 0 & 0 & 0 & a \\ 0 & 0 & 0 & b \\ 0 & 0 & 0 & c \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where a, b and c are how much the position vector should be moved by in the x, y and z axis respectively

Euler Rotations

$$T_x=\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta_x) & -\sin(\theta_x) & 0 \\ 0 & \sin(\theta_x) & \cos(\theta_x) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$T_y = \begin{pmatrix} \cos(\theta_y) & 0 & \sin(\theta_y) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta_y) & 0 & \cos(\theta_y) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$T_z = \begin{pmatrix} \cos(\theta_z) & -\sin(\theta_z) & 0 & 0 \\ \sin(\theta_z) & \cos(\theta_z) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$T = T_y \times (T_z \times T_x)$$

where θ_{axis} is how much the position vector should be rotated about the specified axis.

Perspective

Perspective matrix transformations are special as they create the illusion of depth by moving position vertices in particular ways.

$$T = \begin{pmatrix} \frac{1}{(\tan(\frac{fov}{2}) \times aspect)} & 0 & 0 & 0 \\ 0 & \frac{1}{(\tan(\frac{fov}{2}))} & 0 & 0 \\ 0 & 0 & \frac{-(far+near)}{(far-near)} & \frac{-(2 \times far \times near)}{(far-near)} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

where fov is the desired field of view of the camera,

aspect is the aspect ratio of the window being rendered with,

far is the far clipping plane (where vertices are removed for being to far from the camera)

and near is the near clipping plane (where vertices are removed for being to close to the camera)

2.5 Shader code

OpenGL has some programmable stages in the graphics pipeline. These are programmed using the shaders. In order to add certain functionality (light, depth, textures, etc.) these must be used.

This project only uses one vertex shader and one fragment shader. Real solutions have many more.

Shaders can also be passed variable data from the main program.

In addition, shaders are very interlinked, with each shader being able to send data further down the pipeline, as shown in the example:

Vertex buffer → Vertex shader → Fragment shader → ...

Data goes right

<u>Vertex Shader</u>

The vertex shader tells the OpenGL API where the position of a vertex is and sends data from the vertex buffer to the fragment shader.

This means the vertex shader will also handle the transformations we need to apply to our mesh.

When we send the position vector of a meshes vertex to OpenGL, we must transform it from its relative position to its spacial position using a transformation matrix.

In addition, to transforming it to the spacial position, we must also apply rendering transformations to the position vertices. This transformation moves the position vectors in a way that creates the illusion of depth when they are drawn.

Before the position and normal vectors are sent to the fragment shader, they must also be transformed so they are in the correct position in space when we

calculate various things in the fragment shader. We would not apply rendering transformations to these.

The position vectors must be multiplied by the transformation matrices before being sent to the fragment shader, but the normal vertices have a more complicated procedure to be corrected.

In order to calculate the correct transformed normal, we must calculate the normal matrix of the transformation we wish to apply.

A normal matrix of a transformation is equal to the transposed inverse of that matrix,

$$\left(M^{-1}\right)^T$$

where M is the matrix transformation

This can then be used to transform the normal to the correct position.

```
// Vertex shader
// Data from the outside
in pos-transform
in render-transform

// Data from the vertex buffer
in positionVertex
in normalVertex
in textureVertex

// Data to be sent to the fragment shader
out positionVert
out normalVert
out textureVert

PROCEDURE main()
        gl_Position = render-transform * pos-transform * positionVertex
        // tells OpenGL what the position vector should be

        // the out variables must be redefined by the shader
        positionVert = pos-transform * positionVertex
        // correcting position vector
        normalMat = transpose(inverse(pos-transform))
        normalVert = normalMat * normalVertex
        // correcting normal vector
        textureVert = textureVertex
ENDPROCEDURE
```

Fragment Shader

The fragment shader handles the colouring of meshes rendered by the vertex shader.

Since at a very basic level, light simulation and texturing is just colouring the mesh in various ways, the fragment shader handles both of these tasks.

Texturing

In order to texture a mesh, first, outside of the shader code, the texture to be rendered on the mesh must be binded with an OpenGL function call.

Once the texture has been binded, the fragment shader can access the texture.

Then, the fragment shader needs to correct the texture coordinates as OBJ and OpenGL handle texture coordinates differently.

```
uv = vec2( tex_coords.x, 1 – tex_coords.y )

// correcting texture coordinates
```

Once these are done, the fragment shader has a function call to add the texture to the vertex.

```
texture( image, uv )

// image should be the texture data
```

Lighting

The project will use three different light equations to simulate light. These are diffuse lighting, ambient lighting and specular lighting.

Ambient lighting simulates the background light level from light reflected off of surfaces. Its equation is

$$I_{ambient} = R\,j$$

where I is the light intensity,

R is the reflectivity of a surface (between 0 and 1),

and j is a constant which determines how intense the ambient light is.

Diffuse lighting is the lighting model that simulates how much light reaches a face depending on how it is orientated towards a light source.

The equation for diffuse lighting is

$$I_{diffuse} = R\,k\cos\theta$$

where θ is the angle between the light source and the face

k is how intense the light source is

and I, R are as previously defined

However, we can work out cos(θ) with the normal vertex and the direction the light by using the equation

$$\vec{A}\cdot\vec{B} = |\vec{A}||\vec{B}|\cos\theta$$

So if the vectors we use are unit vectors then

$$\hat{A}\cdot\hat{B} = \cos\theta$$

as the magnitudes of A and B would both be one.

This lets us derive the more useful light equation

$$I_{diffuse} = R\,k\left(\hat{N}\cdot\hat{D}\right)$$

where N is the normal unit vector and D is the direction unit vector of the light source.

Specular lighting simulates how shiny objects reflect light, creating the bright spot seen on these objects.

IMAGE EXAMPLE

The lighting model equation is calculated with

$$I_{specular} = R\,k\cos^n \phi$$

where ϕ is the angle between the direction of the light and the direction of the viewer

n is how shiny we want an object to be

and I, R and k are as previously defined

We can derive a more useful equation the same way as we did last time to create this equation

$$I_{specular} = R\,k\,(\hat{D} \cdot \hat{V})^n$$

where D is as previously defined and V is the direction unit vector of the viewer

Lights only 'emit' diffuse and specular light. In order to calculate the total intensity coming from a light, the values of these intensities just need to be added together.

$$I_{total} = I_{diffuse} + I_{specular}$$

Then, to calculate the light falloff from the light source, the equation

$$I_{light} = I_{total} \times \left(\frac{c}{distance^2}\right)$$

where c is an arbitrary constant and distance is the distance of the object from the light source

This is derived from the phenomenon experienced in real life

$$I \propto \frac{1}{distance^2}$$

Finally, to calculate the light intensity on a mesh, the intensities from each light has to be summed together, then the ambient light intensity added on.

$$I = I_{ambient} + \sum I_{light}$$

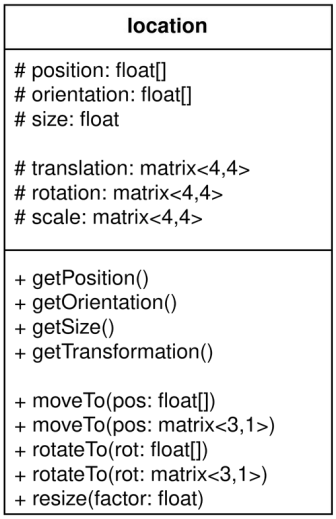
<u>Implementation</u>

The fragment shader will bring the previous two sections together as such:

```
// Data from the outside
in cameraPosition
in lightPositions
in lightIntensities
in surfaceReflectivity
in ambientConstant
in shinyness
in image

// Data from vertex shader
// Variable names must be the same as the 'out' variables in the vertex shaders
// for OpenGL shaders to use them properly
in posVec
in norVec
in texVec

// Send the colour of the vertex to OpenGL
out FragColor

PROCEDURE main()
    ambientIntensity ← surfaceReflectivity * ambientConstant
    cameraDirection ← cameraPosition - positionVector

    intensity ← ambientIntensity * vec(1, 1, 1)
    // light intensities are multiplied by the vector (1, 1, 1) to indicate that a light source is white light
    for i → len( lightPositions ) do
        lightDirection ← lightPosition[i] – positionVector
        lightDistance ← magnitude(lightDirection)

        costheta ← dot(unit(norVec), unit(lightDirection)
        diffuseIntensity ← surfaceReflectivity * lightIntensities[i] * costheta

        cosphi ← dot(unit(lightDirection), unit(cameraDirection)
        specularIntensity ← surfaceReflectivity * lightIntensities[i] * (cosphi ** shinyness)
        lightIntensity ← (5 / lightDistance ** 2) * (specularIntensity + diffuseIntensity)
        intensity ← intensity + lightIntensity * vec(1, 1, 1)
    next i
    FragColor ← intensity * texture(image, UV)
ENDPROCEDURE
```

2.6 Rendering classes

In order to organise the data that is required during the rendering process, there will be various classes made to store the required information.



The location class will be a class other classes can inherit members to store information about where they should be located.

The fields are protected rather than private so subclasses can override some of the functions and still use these variables.

The position, orientation and size members store the values so we can work out where a model should be. These values should only be changed if the associated matrix is also changed to the correct value, hence why they are protected.

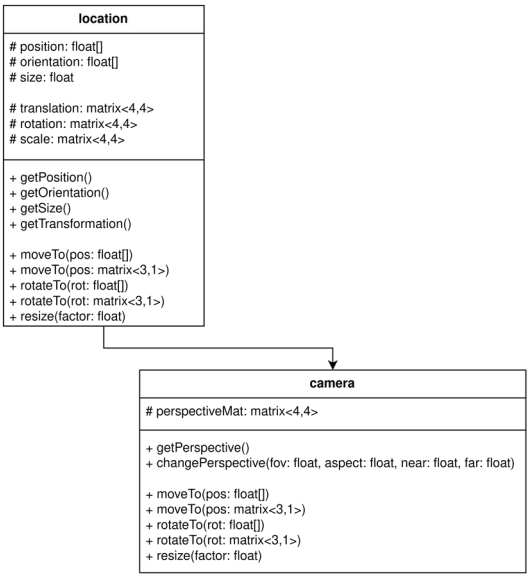
The getPosition(), getOrientation() and getSize() then return a copy of their associated data, meaning we can still use these values without worrying about accidentally editing them when we shouldn't.

The getTransformation() method multiples all the stored transformation matrices stored and returns one big transformation, reducing the amount of matrices we have to deal with.

```
FUNCTION getTransformation()

        return self.translation * self.rotation * self.scale

ENDFUNCTION
```

Although the remaining methods are likely to be overridden by subclasses, they still have default functionality.

These functions update the stored location data, and also update the stored transformation matrices to the new location.



The camera class is one of the subclasses of location. This class will give a global transformation to all visible meshes to simulate the position of view changing.

To do this, the location changing methods are overridden, then the arguments for them are reversed. e.g.,

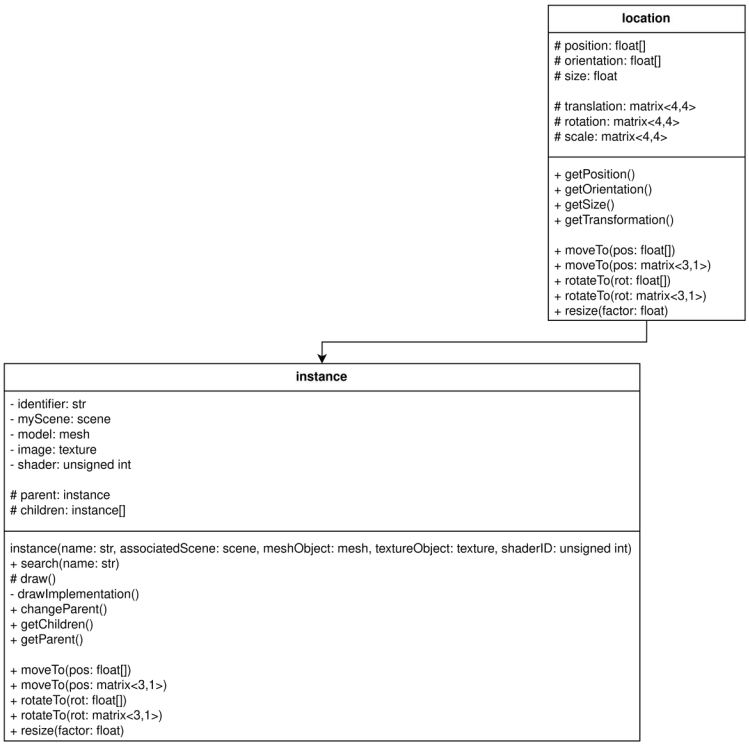
```
PROCEDURE moveTo(position)

        reversedPosition = [ -position[0], -position[1], -position[2] ]

        //create a translation matrix with reversedPosition

        ...

ENDPROCEDURE
```

We do this as from our perspective, if we moved right everything else around us moves left (if it is stationary). This means if we moved the camera in one direction in our program, we should move everything else around us in the opposite direction.

The resize method for the camera is overridden to be empty, as this doesn’t make much sense for a camera.

In addition to providing a global transformation, the camera also stores the perspective matrix.

The changePerspective method lets us change the matrix safely by passing the required values as arguments, and we can get a copy of the matrix with the getPerspective method.



The other subclass of location is instance. This classes main function is storing all the relevant data to render a mesh.

Instances are also stored as a tree, having a parent, children and a root node.

All actions applied to an instance will also be applied to its children (and their children and so on), letting us preform actions to an entire scene by calling a function on the root node. This will be accomplished using a pre-order graph traversal.

e.g., moveTo will be overridden as:

```
PROCEDURE moveTo(pos)
        move( self, pos ) //placeholder to represent moving the instance
        IF len( self.children ) == 0 THEN
                return                  //dont do anything if there are no children
        ENDIF
        FOR i → len( self.children ) DO
                self.children[i].moveTo(pos)
        NEXT i
ENDPROCEDURE
```

Other than including the child propagation, the overridden functions remain the same.

Instances can also be given an identifier that lets us search for a specific instance. Although these identifiers should be unique, this isn't enforced and doesn't cause any trouble if this isn't the case.

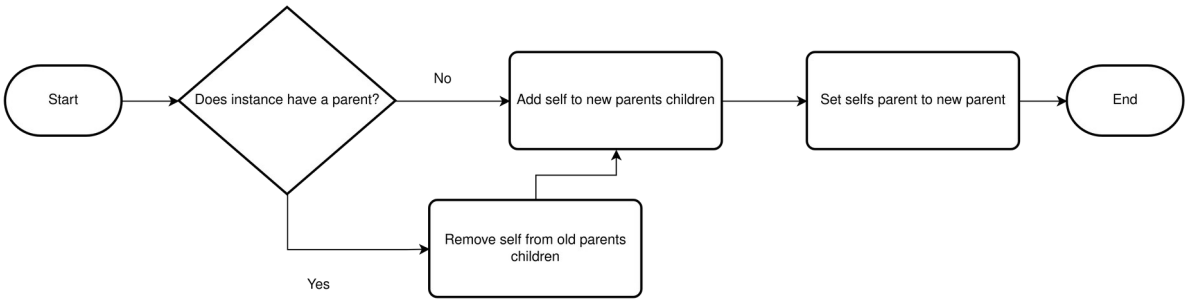
The searching works by preforming a linear search on the pre-order traversal.

```
FUNCTION search(name)
        IF self.identifier == name THEN
                return self
        ENDIF
        IF len( self.children ) == 0 THEN
                return NULL
        ENDIF
        FOR i→ len( self.children ) DO
                result = self.children[i].search(name)
                IF result != NULL THEN
                        return result
                ENDIF
        NEXT i
        return NULL
ENDFUNCTION
```

Instances also need to be able to add and remove children, and change their parent.

Since an instances relationship is recorded on two objects, the parent field and the children field will be protected. A read-only version of this data will be obtained with getParent() and getChildren() respectively.

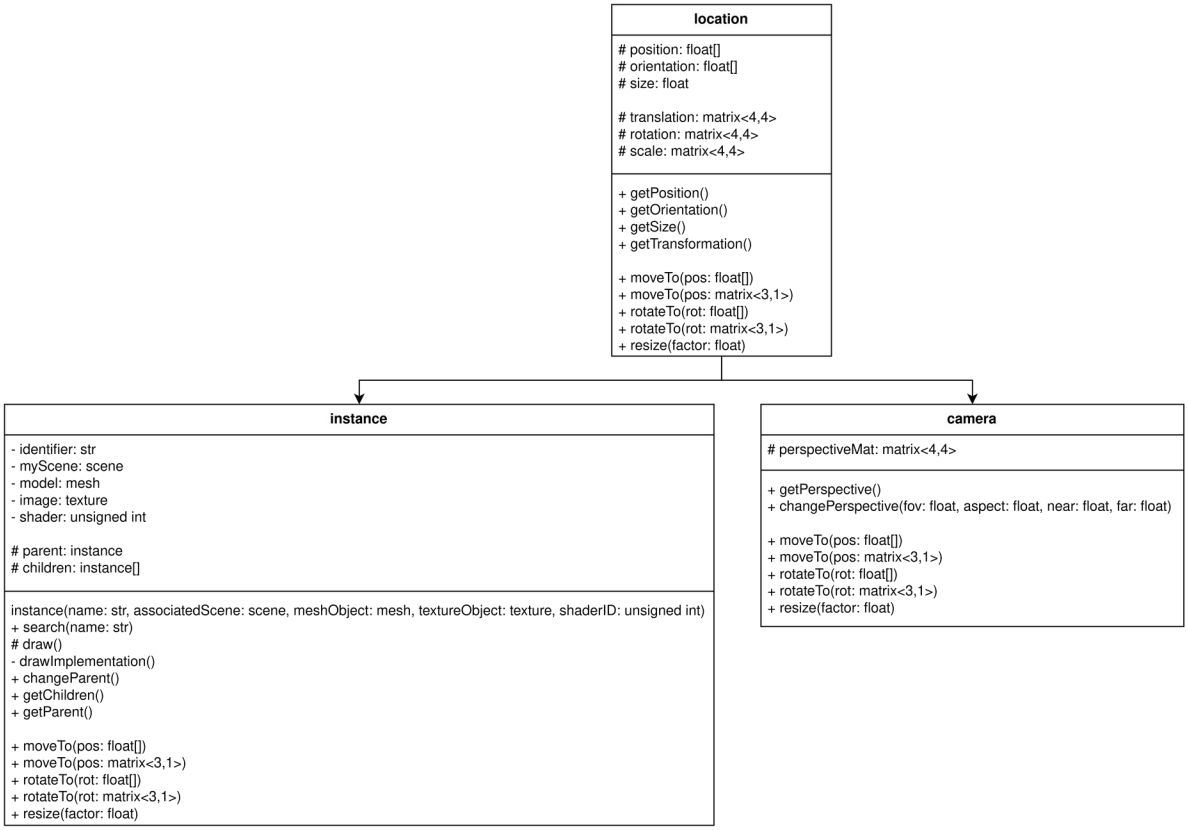
To change an instances parent (which in turn either removes it as a child from its previous parent or adds it as a child to its new parent) the changeParent method can be called.



Since instances may need a different method to draw each one, there is a wrapper function for drawing too. This function just calls the underlying draw implementation, using its function pointer, meaning drawing (and shader code) can be different for every instance even though they come from the same object.

Lastly, the instance constructor just put the associated arguments into the correct fields.

Full class diagram for location and its subclasses:



A scene object brings all the previous classes together.

scene
- window: OpenGL object + currentCamera: camera + root: instance
scene(rootInstance: instance, renderingWindow: OpenGL object) + changeRoot(rootInstance: instance) + drawScene()

It stores the window that will be rendered to, the root node of the instance tree and the camera that should be used by instances.

3. Technical solution

3.1 main.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <curl/curl.h>

#include <iostream>
#include <fstream>
#include <string>
#include <sstream>

#include "graphicHandler.h"
#include "graphicMath.h"
#include "fileHandler.h"
#include "hashtable.h"
#include "licenseChecker.h"

int main()
{
    //licenseProcess();   //license key checking process

    scene newscene;
```

```
    //create hashmap pointing to read functions
    hashtable<readImgPtr, 10> default_imageFormatLoaders = {
        {"png", &readPNG}
    };
    hashtable<readMeshPtr, 10> default_meshFormatLoaders = {
        {"obj", &readOBJ_old}
    };

    formatLoaders formats;
    formats.imgFormats = &default_imageFormatLoaders;
    formats.meshFormats = &default_meshFormatLoaders;

    //create format loader object
    fileLoader newFileloader(formats);

    //load meshes and textures
    mesh monkey = newFileloader.meshLoader("meshes/monkey.obj");
    mesh square = newFileloader.meshLoader("meshes/bettercube.obj");
    texture box = newFileloader.imageLoader("textures/box.png");
    texture checkers =
newFileloader.imageLoader("textures/checkermap_resized.png");

    //load in shader
    shader newShader = newFileloader.shaderLoader({
"shaders/fragmentShader.frag", "shaders/vertexShader.vert" });
```

```
    //create necessary objects for rendering
    camera newCam;
    newCam.changePerspective(toRad(70.0f), 1.0f, 1.0f, 30.0f);

    light newLight;
    newLight.intensity = 10.0f;
    newLight.move({ 5.0f, 5.0f, 5.0f });

    newscene.currentCamera = newCam;
    newscene.lights.push_back(newLight);

    instance instance1("instance1", &newscene, &monkey, &box, &newShader);
    instance1.drawImplementation = &drawImplementation;
    instance1.move({ 0.0f, 0.0f, -6.5f });

    while (!glfwWindowShouldClose(newscene.window)) {    //render loop
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        glClearColor(255.0f / 255.0f, 255.0f / 255.0f, 255.0f / 255.0f, 1.0f);  //set
background colour to white


        instance1.draw();   //draw instance

        instance1.rotate({ 0.0f, static_cast<float>(glfwGetTime()), 0.0f }); //rotates
the object in a circle for the runtime of the application
```

```
        glfwSwapBuffers(newscene.window);   //swap to the updated render buffer
    }

    glfwTerminate();

    return 0;
}
```

3.2 fileHandler

3.2.1 fileHandler.h

```
#pragma once

#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <array>
#include <regex>
#include <initializer_list>
#include <fstream>
#include <sstream>
#include <cmath>
#include <glad/glad.h>
#include <libpng/png.h>

#include "hashtable.h"



//defining structs used by functions
//images
struct image {
	unsigned int width;
```

```
	unsigned int height;
	unsigned char* data;
};

struct texture {
	unsigned int textureID;
	~texture();	//deconstructor for automatic cleanup
};

//meshes
struct mesh {
	unsigned int vbo;
	unsigned int vao;
	unsigned int size;
	~mesh();	//deconstructor for automatic cleanup
};

//shaders
struct shader {
	unsigned int shaderID;
	~shader();	//deconstructor for automatic cleanup
};



//defining our read functions
```

```
//images
image readPNG(std::string filepath);

//meshes
std::vector<float> readOBJ(std::string filepath);
std::vector<float> readOBJ_old(std::string filepath);



//defining default dictionaries
typedef std::vector<float>(*readMeshPtr)(std::string);
typedef image(*readImgPtr)(std::string);

//defining file loading classes
struct formatLoaders {
	hashtable<readImgPtr, 10>* imgFormats;
	hashtable<readMeshPtr, 10>* meshFormats;
};



class fileLoader {
public:
	fileLoader(formatLoaders formatFunctions) : formats{ formatFunctions }
{};

	texture imageLoader(std::string filepath);
```

```
    mesh meshLoader(std::string filepath);
    shader shaderLoader(std::initializer_list<std::string> filepaths);    //can take any amount of filepaths
private:
    formatLoaders formats;
};
```

3.2.2 fileHandler.cpp

```
#include "fileHandler.h"

//deconstructors so opengl buffers are destroyed cleanly
texture::~texture() {
	if (textureID != NULL) {
		glDeleteTextures(1, &textureID);
	}
}

mesh::~mesh() {
	if (vbo != NULL) {
		glDeleteBuffers(1, &vbo);
	}
	if (vao != NULL) {
		glDeleteBuffers(1, &vao);
	}
}

shader::~shader() {
	if (shaderID != NULL) {
		glDeleteProgram(shaderID);
	}
}

// helper function to get the file extension from a file path
```

```
// e.g., returns png for abc.png
static inline std::string getExt(std::string filepath) {
	static const std::regex ext("([a-z]|[A-Z]|[0-9]|/|./|../|_)*.([a-z]*|[A-Z]*)");
	std::smatch match;
	std::regex_match(filepath, match, ext);

	if (match.size() < 2) {
		std::cout << "no file extension\n";
		return "";
	}

	return match[2];
}




image readPNG(std::string filepath) {
	image newImage;
	newImage.data = nullptr;
	newImage.width = 0;
	newImage.height = 0;

	//checks to ensure file being read exists and is a png
	FILE* file = fopen(filepath.c_str(), "rb");
	if (!file) {
		return newImage;
```

```
	}
	unsigned char header[8];
	if (fread(header, 1, 8, file) != 8) {
		return newImage;
	}
	static constexpr unsigned char pngSig[8] = { 137, 80, 78, 71, 13, 10, 26,
10 };
	if (memcmp(header, pngSig, 8) != 0) {
		return newImage;
	}

	//this section was taken from the libpng documentation
	// initialising libpng
	png_structp pngPtr = png_create_read_struct(
		PNG_LIBPNG_VER_STRING,
		NULL,
		NULL,
		NULL
	);
	if (!pngPtr) {
		fclose(file);
		return newImage;
	}
	png_infop infoPtr = png_create_info_struct(pngPtr);
	if (!infoPtr) {
		png_destroy_read_struct(&pngPtr, NULL, NULL);
```

```
        fclose(file);
        return newImage;
    }
    if (setjmp(png_jmpbuf(pngPtr))) {
        png_destroy_read_struct(&pngPtr, &infoPtr, NULL);
        fclose(file);
        return newImage;
    }
    png_init_io(pngPtr, file);
    png_set_sig_bytes(pngPtr, 8);

    //reading png and image resolution
    png_read_png(pngPtr, infoPtr, PNG_TRANSFORM_IDENTITY, NULL);
    unsigned char** rowPtrs = png_get_rows(pngPtr, infoPtr);
    int rowBytes = static_cast<int>(png_get_rowbytes(pngPtr, infoPtr));
    //end of section taken from libpng

    newImage.width = png_get_image_width(pngPtr, infoPtr);
    newImage.height = png_get_image_height(pngPtr, infoPtr);

    //check if the image is too big for opengl
    if (newImage.width * newImage.height > GL_MAX_TEXTURE_SIZE *
GL_MAX_TEXTURE_SIZE) {
        std::cout << "texture has exceeded OpenGL max texture size\n";
        return newImage;
    }
```

```
	unsigned char* data = new unsigned char[rowBytes * newImage.height];
	newImage.data = data;

	// moving image data into buffer
	for (int i = 0; i < newImage.height; ++i) {
		memmove(data + i * rowBytes, rowPtrs[i], rowBytes);
	}

	//cleanup libpng
	png_destroy_read_struct(&pngPtr, &infoPtr, NULL);
	fclose(file);

	return newImage;
}

static void objRegex(std::string line, std::regex reg, std::vector<float>* vec, int
len) {
	std::smatch matchObj;
	std::regex_match(line, matchObj, reg);

	if (matchObj.size() < 2) {	//throw if no value is returned from the regex
		throw "something went wrong";
	}

	for (int i = 0; i < len; ++i) {
```

```
                vec->push_back(std::stof(matchObj[i + 1].str()));       //put the values extracted from the regex into the passed vector
        }
}

std::vector<float> readOBJ_old(std::string filepath) {
        std::vector<float> verticies;
        std::vector<float> vertexPos;
        std::vector<float> vertexNor;
        std::vector<float> vertexTex;

        std::ifstream obj(filepath);
        if (!obj.is_open()) {   //check if the file exists
                return verticies;
        }

        //defining regular expressions
        static const std::regex v("v (-?[0-9]+.[0-9]*) (-?[0-9]+.[0-9]*) (-?[0-9]+.[0-9]*)");
        static const std::regex vn("vn (-?[0-9]+.[0-9]*) (-?[0-9]+.[0-9]*) (-?[0-9]+.[0-9]*)");
        static const std::regex vt("vt (-?[0-9]+.[0-9]*) (-?[0-9]+.[0-9]*)");
        static const std::regex f("f ([0-9]*/[0-9]*/[0-9]*) ([0-9]*/[0-9]*/[0-9]*) ([0-9]*/[0-9]*/[0-9]*)");
        static const std::regex index("([0-9]*)/([0-9]*)/([0-9]*)");

        //for loop for the various datatypes of obj file
```

```
for (std::string line; std::getline(obj, line); ) {
	switch (line.c_str()[0] << sizeof(char) * 8 | line.c_str()[1]) {
	case * "v" << sizeof(char) * 8 | *" ": // 30240
		objRegex(line, v, &vertexPos, 3);

		break;

	case * "v" << sizeof(char) * 8 | *"n": // 30318
		objRegex(line, vn, &vertexNor, 3);

		break;

	case * "v" << sizeof(char) * 8 | *"t": // 30324
		objRegex(line, vt, &vertexTex, 2);



		break;
	case * "f" << sizeof(char) * 8 | *" ": // 26144
		std::smatch matchObj;
		std::regex_match(line, matchObj, f);

		if (matchObj.size() < 2) {
			throw "malformed obj file";
		}

		std::vector<std::string> str_f;
```

```
                    for (int i = 0; i < 3; i++) {
                        str_f.push_back(matchObj[i + 1].str());
                    }

                    for (int i = 0; i < 3; i++) {     //extract data from indicies and
put into vertices
                        std::smatch faceIndices;
                        std::regex_match(str_f[i], faceIndices, index);

        verticies.push_back(vertexPos[(std::stoi(faceIndices[1].str()) - 1) * 3]);

        verticies.push_back(vertexPos[(std::stoi(faceIndices[1].str()) - 1) * 3 + 1]);

        verticies.push_back(vertexPos[(std::stoi(faceIndices[1].str()) - 1) * 3 + 2]);

        verticies.push_back(vertexTex[(std::stoi(faceIndices[2].str()) - 1) * 2]);

        verticies.push_back(vertexTex[(std::stoi(faceIndices[2].str()) - 1) * 2 + 1]);

        verticies.push_back(vertexNor[(std::stoi(faceIndices[3].str()) - 1) * 3]);

        verticies.push_back(vertexNor[(std::stoi(faceIndices[3].str()) - 1) * 3 + 1]);

        verticies.push_back(vertexNor[(std::stoi(faceIndices[3].str()) - 1) * 3 + 2]);
                    }
```

```
                    break;
            }
        }

        return verticies;
}



texture fileLoader::imageLoader(std::string filepath) {
        texture newTexture;
        std::string fileExt = getExt(filepath);

        hashobject<readImgPtr>* imgFormat = formats.imgFormats-
>find(fileExt);         //get function from hashtable
        if (imgFormat == nullptr) {
                std::cout << "file format not supported\n";
                return newTexture;
        }

        image newImage = imgFormat->value(filepath);      //grab image data

        if (newImage.data == nullptr) {   //error has happened when loading
image
                return newTexture;
        }
```

```
	//create opengl texture id
	glGenTextures(1, &newTexture.textureID);
	glBindTexture(GL_TEXTURE_2D, newTexture.textureID);

	//basic texture setup
	glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
	glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
	glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
	glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

	//send image data to opengl api
	glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, newImage.width,
newImage.height, 0, GL_RGB, GL_UNSIGNED_BYTE, newImage.data);

	//unbind texture
	glGenerateMipmap(GL_TEXTURE_2D);
	glBindTexture(GL_TEXTURE_2D, 0);

	//delete image data now that image is loaded by opengl
	delete[newImage.width * newImage.height] newImage.data;
	newImage.data = nullptr;

	return newTexture;
}
```

```
mesh fileLoader::meshLoader(std::string filepath) {
	mesh newMesh;
	std::string fileExt = getExt(filepath);
	hashobject<readMeshPtr>* meshFormat = formats.meshFormats->find(fileExt);	//get function from hashtable
	if (meshFormat == nullptr) {
		std::cout << "unsupported file format\n";
		return newMesh;
	}
	std::vector<float> vertices = meshFormat->value(filepath);	//get the verticies from the function

	if (vertices.size() == 1) {
		return newMesh; //failed to read obj file
	}

	//create vao and vbo
	glGenVertexArrays(1, &newMesh.vao);
	glBindVertexArray(newMesh.vao);
	glGenBuffers(1, &newMesh.vbo);
	glBindBuffer(GL_ARRAY_BUFFER, newMesh.vbo);

	glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.data(), GL_STATIC_DRAW);	//put data into the vbo
```

```
	//create vertex array object
	//tells opengl what values mean depending on where they are in the vbo
	glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), 0); // positions
	glEnableVertexAttribArray(0);

	glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float))); // textures
	glEnableVertexAttribArray(1);

	glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(5 * sizeof(float))); // normals
	glEnableVertexAttribArray(2);

	//unbind vao and vbo
	glBindVertexArray(0);
	glBindBuffer(GL_ARRAY_BUFFER, 0);

	newMesh.size = vertices.size();
	return newMesh;
}

//compiles shader parts and returns the opengl id
static unsigned int compileShader(std::string filepath, GLenum type) {
	std::ifstream text(filepath);
	if (!text.is_open()) {
```

```
        std::cout << filepath << "\ndoesn't exist\n";
        return 0;
    }

    std::stringstream buffer;
    buffer << text.rdbuf();
    std::string temp = buffer.str();
    const char* shaderText = temp.c_str(); //extract c string version of the file text
    text.close();

    unsigned int shaderpart = glCreateShader(type);
    glShaderSource(shaderpart, 1, &shaderText, NULL);      //send shaderpart text to opengl for compilation
    glCompileShader(shaderpart);   //compile shader

    int result;
    glGetShaderiv(shaderpart, GL_COMPILE_STATUS, &result);      //check if shader compiled properly
    if (!result) {   //output error and cleanup if it didnt
        char errLog[512];
        glGetShaderInfoLog(shaderpart, 512, NULL, errLog);
        std::cout << errLog << "\n";
        glDeleteShader(shaderpart);
        return 0;
    }
```

```
	return shaderpart;
};

shader fileLoader::shaderLoader(std::initializer_list<std::string> filepaths) {
	std::map<std::string, int> shaderParts = {	//dictionary containing flags if that shader type has been loaded
		{"vert", 0},
		{"frag", 0},
		{"tess", 0},
		{"geom", 0}
	};
	std::map<std::string, GLenum> types = {	//dictionary to get the correct glenum when compiling shader
		{"vert", GL_VERTEX_SHADER},
		{"frag", GL_FRAGMENT_SHADER},
		{"tess", GL_TESS_CONTROL_SHADER},
		{"geom", GL_GEOMETRY_SHADER}
	};

	shader newShader;
	newShader.shaderID = glCreateProgram();

	for (auto it = filepaths.begin(); it != filepaths.end(); ++it) {
		std::string fpExt = getExt(*it);
```

```
		auto spIt = shaderParts.find(fpExt);	//get iterator of key so we
dont have to query for it multiple times
		if (spIt == shaderParts.end()) {	//file extension isnt a recognised
shader
			continue;
		}
		if (spIt->second) {	//shader of that type already loaded for shader
program
			continue;
		}

		unsigned int shaderPart = compileShader(*it, types[fpExt]);
	//compile the shader
		if (shaderPart == 0) {		//an error occured if the returned value is
zero
			continue;
		}

		spIt->second = shaderPart;
		glAttachShader(newShader.shaderID, shaderPart);
	}

	glLinkProgram(newShader.shaderID);
	for (auto it = shaderParts.begin(); it != shaderParts.end(); ++it) {	//delete
all useless shaders
		if (it->second) {
			glDeleteShader(it->second);
```

```
            }
        }

        return newShader;
}
```

3.3 licenseChecker

3.3.1 licenseChecker.h

```
#pragma once

#include <curl/curl.h>
#include <Windows.h>

#include <string>
#include <iostream>

int checkLicense(std::string license, std::string email);

void licenseProcess();
```

3.3.2 licenseChecker.cpp

```
#include "licenseChecker.h"


int checkLicense(std::string license, std::string email) {
    if (license == "" || email == "") { //ensure license or email isnt empty
        return 0;
    }


    //format license and email into json
    std::string json =
        "{\n"
        " \"license\": \"" + license + "\",\n"
        " \"email\": \"" + email + "\"\n"
        "}\n";



    //setup libcurl to send json
    CURL* curl;
    CURLcode result;
    long response = 0;

    result = curl_global_init(CURL_GLOBAL_ALL);
    if (result) {
        return 0;
    }
```

```
    curl = curl_easy_init();
    curl_slist* header = NULL;
    if (curl) {
        header = curl_slist_append(header, "Content-Type: text/plain");

        curl_easy_setopt(curl, CURLOPT_URL, "http://127.0.0.255/licenses"); //ip and application the webserver is hosted on
        curl_easy_setopt(curl, CURLOPT_POSTFIELDS, json.c_str());
        curl_easy_setopt(curl, CURLOPT_HTTPHEADER, header);
        result = curl_easy_perform(curl);

        if (result == CURLE_OK) {
            curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &response); //get response code
        }
        else {
        }
        curl_easy_cleanup(curl);
    }
    curl_global_cleanup();

    return static_cast<int>(response);  //return response code from server
}

void licenseProcess() {
    while (true) {
```

```
        MessageBox(NULL, (LPCWSTR)L"Enter your license key into the console.", (LPCWSTR)L"License", NULL);
        std::string license;
        std::cin >> license;

        MessageBox(NULL, (LPCWSTR)L"Now enter your email into the console.", (LPCWSTR)L"License", NULL);
        std::string email;
        std::cin >> email;

        int respone = checkLicense(license, email);
        if (respone == 200) {   //server will send back 200 OK if response is valid
            MessageBox(NULL, (LPCWSTR)L"Valid license key and email", (LPCWSTR)L"License", NULL);
            return;
        }

        int msgboxID = MessageBox(NULL, (LPCWSTR)L"Invalid license key or email, try again?", (LPCWSTR)L"License", MB_ICONWARNING | MB_RETRYCANCEL | MB_DEFBUTTON2);
        if (msgboxID == 2) {    //user clicked cancel with invalid license, so stop program
            exit(0);
        }
    }
}
```

3.3.3 server.py

```
import json

import re

import sqlite3

from http.server import BaseHTTPRequestHandler, HTTPServer


def licenseHandler(license : str, email : str): # cross references license against database, returns a http response code

    licenseRe = "[A-Z][A-Z][A-Z][A-Z]-[A-Z][A-Z][A-Z][A-Z]-[A-Z][A-Z][A-Z][A-Z]-[A-Z][A-Z][A-Z][A-Z]"

    emailRe = "([a-zA-Z]*|[0-9]*)@([a-zA-Z]*|[0-9]*).com"


        return 200


    # sanatize inputs

    match = re.fullmatch(licenseRe, license)

    if match == None:

        return 400

    match = re.fullmatch(emailRe, email)

    if match == None:

        return 400



    db = sqlite3.connect("licenses.db")

    cursor = db.cursor()
```

```
    res = cursor.execute("SELECT license FROM validLicenses")
    licenseTable = res.fetchall()
    flag = False
    for i in licenseTable: # check if license exists
        if i[0] == license:
            flag = True
            break

    if flag == False:
        return 401 # license doesnt exist

    res = cursor.execute(f"SELECT email FROM validLicenses WHERE license =
'{license}'")
    emailTable = res.fetchone()
    if emailTable[0] == '':
        cursor.execute(f"UPDATE validLicenses SET email = '{email}' WHERE license
= '{license}'")
        db.commit()
        return 200 # unclaimed license now owned by user
    elif emailTable[0] == email:
        return 200 # provided license key is owned by user

    return 401 # license is not owned by user


class server(BaseHTTPRequestHandler):
```

```
    def do_POST(self):
        match self.path:
            case "/licenses": # license checker resource
                data = self.rfile.read(int(self.headers["Content-Length"])).decode() # gets the posted data
                try:
                    info = json.loads(data) # parse posted data as json
                    response = licenseHandler(info["license"], info["email"])
                    if response == 400:
                        raise Exception("bad input")

                    self.send_response(response)
                    self.send_header("Content-type", "application/json")
                    self.end_headers()
                except (json.decoder.JSONDecodeError, KeyError, Exception): # malformed data recieved
                    self.send_response(400)
                    self.send_header("Content-type", "application/json")
                    #self.wfile.write("malformed data sent".encode())
                    self.end_headers()
                except: # unknown error
                    self.send_response(500) # assume some server-side error happened
                    self.send_header("Content-type", "application/json")
                    #self.wfile.write("unknown error occurred".encode())
                    self.end_headers()
```

```
            case _: # default case; resource doesnt exist
                self.send_response(404)
                self.send_header("Content-type", "")
                self.end_headers()



if __name__ == "__main__":
    try: # start server
        server = HTTPServer(('127.0.0.255', 80), server)
        server.serve_forever()
    except KeyboardInterrupt: # stop server gracefully when a keyboard
interrupt happens
        server.socket.close()
        print("stopping...")
```

3.3.4 keyCreator.py

```
import sqlite3
import random
import string

db = sqlite3.connect("licenses.db")
cursor = db.cursor()

# check if table exists in db, create one if it doesnt
res = cursor.execute("SELECT name FROM sqlite_master")
if (len(res.fetchall()) != 1):
    cursor.execute("CREATE TABLE validLicenses(license, email)")

while True:
    try:
        choice = int(input("1. Create new license key \n2. Print table\n3. Clear
table\n"))
    except KeyboardInterrupt: # exit gracefully if a keyboard interrupt happens
        break
    except ValueError:
        print("Choose a number")
```

```
    match choice:
        case 1: # add new license
            while True:
                newLicense = ""
                for i in range(4): # random generate a new license
                    for j in range(4):
                        newLicense = newLicense + random.choice(string.ascii_uppercase)
                    if i != 3:
                        newLicense = newLicense + "-"

                res = cursor.execute("SELECT license FROM validLicenses")
                licenses = res.fetchall()

                licenseTable = res.fetchall()
                flag = False
                for i in licenseTable: # check if license already exists in table
                    if i[0] == license:
                        flag = True
                        break
                if flag == True: # generate a new license if it does
                    continue

                print(newLicense)
                cursor.execute(f"INSERT INTO validLicenses VALUES ('{newLicense}',
'')")
                db.commit()
```

```
            break
        case 2: # loop through rows and print each row
            for r in cursor.execute("SELECT license, email FROM validLicenses"):
                print(r)
        case 3: # clear & recreate table
            cursor.execute("DROP TABLE validLicenses")
            cursor.execute("CREATE TABLE validLicenses(license, email)")
            db.commit()
        case _: # default case; invalid input
            print("Choose a valid number")


    print("\n")
```

3.4 hashtable

3.4.1 hashtable.h

```
#pragma once

#include <string>
#include <iostream>
#include <vector>
#include <initializer_list>
#include <utility>
#include <optional>

int hash(std::string value, int s);

template<typename valType>
class hashobject {   //class for linked list in the hashtable
public:
	std::string key = "";
	valType value;
	hashobject* child = nullptr;
	hashobject* parent = nullptr;
};

template<typename valType, int s>
class hashtable {
private:
	std::vector< hashobject<valType>* > table;
```

```
	const int size = s;
public:
	inline hashtable() { table.resize(s); };	//constructor with no agruments
for empty initialisation


	inline hashtable(std::initializer_list< std::pair<std::string, valType> >
newTable) {	//takes a list with the key and value pairs
		table.resize(s);	//resize array to correct size
		for (auto it = newTable.begin(); it != newTable.end(); ++it) {
	//iterate through the passed tables values
			this->add(*it);	//call the add function as to not repeat
code
		}
	};


	inline void add(std::pair<std::string, valType> newPair) {
		hashobject<valType>* newhash = new hashobject<valType>;
	//create a hashobject on the heap for linked list


		//set the objects key and value to correct data
		newhash->key = newPair.first;
		newhash->value = newPair.second;


		int strhash = hash(newPair.first, s);	//get the keys hash value


		if (table[strhash] == nullptr) {	//if no collision set the new key
value pair to that position
```

```
                table[strhash] = newhash;
            }
            else {  //if there is a collision iterate down the chain until the
bottom, then add hashobject at the bottom
                hashobject<valType>* parent = nullptr;
                hashobject<valType>* child = table[strhash];
                while (child != nullptr) {
                    parent = child;
                    child = parent->child;
                }
                newhash->parent = parent;
                parent->child = newhash;
            }
        };

        inline hashobject<valType>* find(std::string key) {
            int strhash = hash(key, s); //turn passed key into hash
            hashobject<valType>* it = table[strhash];
            while (it != nullptr) {
                if (it->key == key) {  //check if the hashobjects key matches
the passed key
                    return it;
                }
                it = it->child; //iterate down the linked list
            }
```

```
            return nullptr;     //return nothing if reach the bottom of chain without finding matching key
        }
};
```

3.4.2 hashtable.cpp

```
#include "hashtable.h"

int hash(std::string val, int s) {
    int total = 0;
    for (auto it = val.begin(); it != val.end(); ++it) {  //add all the decimal values of the characters together to create hash
        total += static_cast<int>(*it);
    }
    return total % s;
}
```

3.5 graphicsMaths

3.5.1 graphicsMath.h

```
#pragma once
#include <iostream>
#include <initializer_list>
#include <cmath>

#define PI 3.14159265359          //define constant
#define TORADFACTOR 0.01745329251

template <int h, int w>
struct matrix {
	float array[h * w] = {};
	static constexpr int height = h; // so other matrices can be created using width and height
	static constexpr int width = w;
	bool identity = false;

	inline matrix() {};

	inline matrix(std::initializer_list<float> init) {
		if (init.size() <= h * w) {	//check if passed array is larger than the matrix
```

```
                std::move(init.begin(), init.end(), array); // move passed
matrix data into the matrices storage
            }
            else { //if it is stop buffer overflow by only reading values in matrix
range
                std::cout << "an initalizer list provided is larger than matrix
size" << "\n";
                std::move(init.begin(), init.begin() + h * w, array);
            }
        };
        inline matrix(float init[h*w]) {
            std::move(init, init + h * w, array);       //if c style array passed
move all values in it to matrices array
        };


        template<class T>
        inline auto operator* (T other) {
            static_assert(width == other.height, "attempted to multiply 2
incompatable matrices");
            //compiler error avoids a runtime crash due to a programmer error


            //skip cases to avoid unnecessary computation
            if (this->identity == true) {
                return other;
            }
            else if (other.identity == true) {
                return *this;
```

```
		}

		matrix<h, other.width> newMatrix;

		for (int k = 0; k < h; ++k) {  //multiply matrices together using
algorithm
			for (int j = 0; j < width; ++j) {
				for (int i = 0; i < other.width; ++i) {
					newMatrix.array[i + k * other.width] += array[j +
k * width] * other.array[i + j * other.width];
				}
			}
		}

		return newMatrix;
	};

	template<class T>
	inline auto operator+ (T other) {
		static_assert(width == other.width && height == other.height,
"attempted to add matricies of different dimensions");

		matrix<h, w> newMatrix;

		for (int i = 0; i < h * w; ++i) {
			newMatrix.array[i] = this->array[i] + other.array[i];   //iterate
through matrix and add together all values
```

```
		}

		return newMatrix;
	}
};

//definitions for simpler use in code
typedef matrix<4, 4> mat4;
typedef matrix<4, 1> vec4;
typedef matrix<3, 1> vec3;

//quick way to get identity matrix
inline mat4 iden4() {
	mat4 newMatrix = {
		1.0f, 0.0f, 0.0f, 0.0f,
		0.0f, 1.0f, 0.0f, 0.0f,
		0.0f, 0.0f, 1.0f, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	newMatrix.identity = true;
	return newMatrix;
};

namespace graphics {
	mat4 createEulerRotation(float x, float y, float z);
	mat4 createEulerRotation(float rot[3]);
```

```
    mat4 createTranslation(float x, float y, float z);
    mat4 createTranslation(float pos[3]);
    mat4 createPerspective(float fov, float aspect, float far, float near);
    mat4 createScale(float factor);
};

inline float toRad(float degrees) { return static_cast<float>(degrees *
TORADFACTOR); }
```

3.5.2 graphicMath.cpp

```
include "graphicMath.h"

/*
all functions here return their respective transformations
there is no simpler way to do this so its manually done
*/

mat4 graphics::createEulerRotation(float x, float y, float z) {
	if (x == 0.0f && y == 0.0f && z == 0.0f) {
		return iden4();
	}

	float cosX = cosf(x);
	float sinX = sinf(x);
	float cosY = cosf(y);
	float sinY = sinf(y);
	float cosZ = cosf(z);
	float sinZ = sinf(z);

	matrix<4, 4> rotX = {
		1.0f, 0.0f, 0.0f, 0.0f,
		0.0f, cosX, -sinX, 0.0f,
		0.0f, sinX, cosX, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
```

```
	matrix<4, 4> rotY = {
		cosY, 0.0f, sinY, 0.0f,
		0.0f, 1.0f, 0.0f, 0.0f,
		-sinY, 0.0f, cosY, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	matrix<4, 4> rotZ = {
		cosZ, -sinZ, 0.0f, 0.0f,
		sinZ, cosZ, 0.0f, 0.0f,
		0.0f, 0.0f, 1.0f, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	return rotY * (rotX * rotZ);
}
mat4 graphics::createEulerRotation(float rot[3]) {
	float x = rot[0];
	float y = rot[1];
	float z = rot[2];
	if (x == 0.0f && y == 0.0f && z == 0.0f) {
		return iden4();
	}

	float cosX = cosf(x);
	float sinX = sinf(x);
	float cosY = cosf(y);
	float sinY = sinf(y);
```

```
	float cosZ = cosf(z);
	float sinZ = sinf(z);

	matrix<4, 4> rotX = {
		1.0f, 0.0f, 0.0f, 0.0f,
		0.0f, cosX, -sinX, 0.0f,
		0.0f, sinX, cosX, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	matrix<4, 4> rotY = {
		cosY, 0.0f, sinY, 0.0f,
		0.0f, 1.0f, 0.0f, 0.0f,
		-sinY, 0.0f, cosY, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	matrix<4, 4> rotZ = {
		cosZ, -sinZ, 0.0f, 0.0f,
		sinZ, cosZ, 0.0f, 0.0f,
		0.0f, 0.0f, 1.0f, 0.0f,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	return rotY * (rotX * rotZ);
}

mat4 graphics::createPerspective(float fov, float aspect, float near, float far) {
	float f = 1.0f / tanf(fov / 2.0f);
```

```
	matrix<4, 4> persp = {
		f / aspect, 0.0f, 0.0f, 0.0f,
		0.0f, f, 0.0f, 0.0f,
		0.0f, 0.0f, -(far + near) / (far - near), -(2 * far * near) / (far - near),
		0.0f, 0.0f, -1.0f, 0.0f
	};
	return persp;
}

mat4 graphics::createTranslation(float x, float y, float z) {
	if (x == 0.0f && y == 0.0f && z == 0.0f) {
		return iden4();
	}

	matrix<4, 4> trans = {
		1.0f, 0.0f, 0.0f, x,
		0.0f, 1.0f, 0.0f, y,
		0.0f, 0.0f, 1.0f, z,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	return trans;
}
mat4 graphics::createTranslation(float pos[3]) {
	float x = pos[0];
	float y = pos[1];
	float z = pos[2];
```

```
	if (x == 0.0f && y == 0.0f && z == 0.0f) {
		return iden4();
	}

	matrix<4, 4> trans = {
		1.0f, 0.0f, 0.0f, x,
		0.0f, 1.0f, 0.0f, y,
		0.0f, 0.0f, 1.0f, z,
		0.0f, 0.0f, 0.0f, 1.0f
	};
	return trans;
}

mat4 graphics::createScale(float factor) {
	if (factor == 1.0f) {
		return iden4();
	}

	matrix <4, 4> scale = {
		factor, 0.0f, 0.0f, 0.0f,
		0.0f, factor, 0.0f, 0.0f,
		0.0f, 0.0f, factor, 0.0f,
		0.0f, 0.0f, 0.0f, factor
	};
	return scale;
}
```

3.6 graphicHandler

3.6.1 graphicHandler.h

```
#pragma once

#define _CRT_SECURE_NO_WARNINGS

#include "graphicMath.h"
#include "fileHandler.h"

#include <glad/glad.h>
#include <libpng/png.h>
#include <glfw/glfw3.h>

#include <iostream>
#include <string>
#include <fstream>
#include <sstream>
#include <cassert>
#include <vector>
#include <array>
#include <regex>
#include <map>

//superclass that has the required members and functions for location data
class location {
```

```
protected:
    float position[3] = { 0.0f, 0.0f, 0.0f };
    float orientation[3] = { 0.0f, 0.0f, 0.0f };
    float size = 1.0f;

    mat4 translation = iden4();
    mat4 rotation = iden4();
    mat4 scale = iden4();
public:
    inline vec3 getPosition() {
        vec3 vector = position;
        return vector;
    };
    virtual void move(std::initializer_list<float> pos);
    virtual void move(vec3 pos);
    inline vec3 getOrientation() {
        vec3 vector = orientation;
        return vector;
    };
    virtual void rotate(std::initializer_list<float> rot);
    virtual void rotate(vec3 rot);
    inline float getSize() {
        return size;
    };
    virtual void resize(float factor);
```

```
    mat4 getTransformation();
};

class camera : public location {    //class used to represent the cameras location
also stores perspective
public:
    //overrides default location functions to reverse input values
    //this makes it act like a camera
    void rotate(std::initializer_list<float> rot) override;
    void rotate(vec3 rot) override;
    void move(std::initializer_list<float> pos) override;
    void move(vec3 rot) override;
    void resize(float factor) override;

    inline mat4 getPerspective() {
        return perspective;
    }
    void changePerspective(float fov, float aspect, float n, float f);  //replaces
perspective matrix if we want to change it
private:
    mat4 perspective;
};

class light : public location { //stores a lights data
public:
    float intensity = 1.0f; //the intensity of the light
```

```
};

static void error_callback(int error, const char* desc) {
    std::cout << error << " " << desc << std::endl;
}

class scene {   //creates the window and stores the lights, camera and ambient
constant
public:
    scene();

    camera currentCamera;
    std::vector<light> lights;
    float ambientConstant = 0.6f;
    GLFWwindow* window;
};

class instance : public location {  //stores all the relevant mesh data so things
can be directly rendered from it
private:
    std::string identifier; //name of the instance so it can be searched for

    //model data
    scene* myScene;
    mesh* myMesh;
    texture* myTexture;
```

```
    shader* myShader;

    float shinyness;
    float reflectivity;

protected:
    instance* parent = nullptr;
    std::vector<instance*> children;

public:
    inline instance(std::string name, scene* scene, mesh* model, texture* img,
shader* shad, float shiny = 2.0f, float reflect = 1.0f) {
        identifier = name;
        myScene = scene;
        myMesh = model;
        myTexture = img;
        myShader = shad;
        shinyness = shiny;
        reflectivity = reflect;
    };

    instance* search(std::string name);

    //drawing functions
    void draw();
```

```
    void (*drawImplementation)(instance*, scene*, mesh*, texture*, shader*, float, float) = nullptr;    //lets us give different draw implementations to instances


    //child / parent tree functions
    inline const std::vector<instance*> getChildren() { return static_cast<const std::vector<instance*>>(children); };
    inline instance* getParent() { return parent; };


    void changeParent(instance* newParent);


    //movement functions
    void rotate(std::initializer_list<float> rot) override;
    void rotate(vec3 rot) override;
    void move(std::initializer_list<float> pos) override;
    void move(vec3 pos) override;
    void resize(float factor) override;
};


void drawImplementation(instance* self, scene* myScene, mesh* myMesh, texture* myTexture, shader* myShader, float shinyness, float reflectivity); //default draw implementation
```

3.6.2 graphicHandler.cpp

```
#include "graphicHandler.h"



instance* instance::search(std::string name) {
	if (identifier == name) {	//return if it is the identifier
		return this;
	}
	//iterates through the children of the root of the search and then their children until it finds a match or runs out of instances
	for (auto it = children.begin(); it != children.end(); ++it) {
		instance* child = *it;
		instance* found = child->search(name);
		if (found != nullptr) {
			return found;
		}
	}
	return nullptr;		//return nothing if everything is searched through
}


void drawImplementation(instance* self,scene* myScene, mesh* myMesh, texture* myTexture, shader* myShader, float shiny, float reflect) {
	if (myMesh->vbo != 0) {
		glUseProgram(myShader->shaderID);	//set correct shader to be used
```

```
//only bind a texture if there is one
if (myTexture->textureID != 0) {
	glBindTexture(GL_TEXTURE_2D, myTexture->textureID);
}

//bind the correct vao and vbo before doing anything
glBindVertexArray(myMesh->vao);
glBindBuffer(GL_ARRAY_BUFFER, myMesh->vbo);

//get locations of the uniforms in our shaders
static const int lightPositions = glGetUniformLocation(myShader->shaderID, "lightPositions");
static const int lightIntensities = glGetUniformLocation(myShader->shaderID, "lightIntensities");
static const int cameraPos = glGetUniformLocation(myShader->shaderID, "cameraPosition");
static const int ambient = glGetUniformLocation(myShader->shaderID, "ambient");
static const int trans = glGetUniformLocation(myShader->shaderID, "trans");
static const int cam = glGetUniformLocation(myShader->shaderID, "cam");
static const int proj = glGetUniformLocation(myShader->shaderID, "proj");

//get required matrices
mat4 transformation = self->getTransformation();
mat4 camera = myScene->currentCamera.getTransformation();
```

```
		mat4 perspective = myScene->currentCamera.getPerspective();


		//send required data to the shaders
		glUniform3fv(lightPositions, 1, myScene->lights[0].getPosition().array);
		glUniform1f(lightIntensities, myScene->lights[0].intensity);
		glUniform1f(ambient, myScene->ambientConstant);
		glUniform3fv(cameraPos, 1, myScene->currentCamera.getPosition().array);
		glUniformMatrix4fv(trans, 1, true, transformation.array);
		glUniformMatrix4fv(cam, 1, true, camera.array);
		glUniformMatrix4fv(proj, 1, true, perspective.array);


		//actually draw
		glDrawArrays(GL_TRIANGLES, 0, myMesh->size);
		glBindTexture(GL_TEXTURE_2D, 0);
	}
}


void instance::draw() {
	//if there is no drawImplementation check parent for one
	if (drawImplementation == nullptr) {
		if (this->parent == nullptr) {
			std::cout << "could not find a draw implementation\n";
			return;
		}
```

```
		if (this->parent->drawImplementation == nullptr) {
			std::cout << "could not find a draw implementation\n";
			return;
		}
	}

	drawImplementation(this, myScene, myMesh, myTexture, myShader,
shinyness, reflectivity);	//actually draw

	//iterate through children and tell them to draw themselves
	if (children.size() == 0) {
		return;
	}
	for (auto it = children.begin(); it != children.end(); ++it) {
		instance* child = *it;
		child->draw();
	}
}


void instance::changeParent(instance* newParent) {
	if (parent != nullptr) {	//remove self from old parents children
		for (auto it = parent->children.begin(); it != parent->children.end(); +
+it) {
			if (*it == this) {
				parent->children.erase(it);
```

```
                    break;
                }
            }
        }
        newParent->children.push_back(this);  //add self to new parents children
        parent = newParent;         //set new parent to parent value
}

/*
these are a lot of methods that just update the location values in a class
they all then create a matrix for the respective transformation
*/

//adds recursion to the functions so that any position update propogates down the tree
void instance::rotate(std::initializer_list<float> rot) {
        if (rot.size() != 3) {
            return;
        }
        std::move(rot.begin(), rot.end(), orientation);
        rotation = graphics::createEulerRotation(orientation);

        if (children.size() == 0) {
            return;
        }
        for (auto it = children.begin(); it != children.end(); ++it) {
```

```
		instance* child = *it;
		child->rotate(rot);
	}
}

void instance::rotate(vec3 rot) {
	std::move(rot.array, rot.array + 3, orientation);
	rotation = graphics::createEulerRotation(orientation);

	if (children.size() == 0) {
		return;
	}
	for (auto it = children.begin(); it != children.end(); ++it) {
		instance* child = *it;
		child->rotate(rot);
	}
}

void instance::move(std::initializer_list<float> pos) {
	if (pos.size() != 3) {
		return;
	}
	std::move(pos.begin(), pos.end(), position);
	translation = graphics::createTranslation(position);

	if (children.size() == 0) {
```

```
		return;
	}
	for (auto it = children.begin(); it != children.end(); ++it) {
		instance* child = *it;
		child->move(pos);
	}
}

void instance::move(vec3 pos) {
	std::move(pos.array, pos.array + 3, position);
	translation = graphics::createTranslation(position);

	if (children.size() == 0) {
		return;
	}
	for (auto it = children.begin(); it != children.end(); ++it) {
		instance* child = *it;
		child->move(pos);
	}
}

void instance::resize(float factor) {
	size = factor;
	scale = graphics::createScale(factor);

	if (children.size() == 0) {
```

```
		return;
	}
	for (auto it = children.begin(); it != children.end(); ++it) {
		instance* child = *it;
		child->resize(factor);
	}
}



//reverses argument inputs so moving camera moves everything in the correct direction
void camera::rotate(std::initializer_list<float> rot) {
	if (rot.size() != 3) {
		std::move(rot.begin(), rot.end(), orientation);
		rotation = graphics::createEulerRotation(-orientation[0], -orientation[1], -orientation[2]);
	}
}

void camera::rotate(vec3 rot) {
	std::move(rot.array, rot.array + 3, orientation);
	rotation = graphics::createEulerRotation(-orientation[0], -orientation[1], -orientation[2]);
}

void camera::move(std::initializer_list<float> pos) {
```

```
	if (pos.size() == 3) {
		std::move(pos.begin(), pos.end(), position);
		translation = graphics::createTranslation(-position[0], -position[1],
-position[2]);
	}
}

void camera::move(vec3 pos) {
	std::move(pos.array, pos.array + 3, position);
	translation = graphics::createTranslation(-position[0], -position[1],
-position[2]);
}

void camera::resize(float factor) {
	size = factor;
	scale = graphics::createScale(-factor);
}

void camera::changePerspective(float fov, float aspect, float n, float f) {
	perspective = graphics::createPerspective(fov, aspect, n, f);
}


//default location functions
void location::rotate(std::initializer_list<float> rot) {
	if (rot.size() == 3) {
```

```
		std::move(rot.begin(), rot.end(), orientation);
		rotation = graphics::createEulerRotation(orientation);
	}
}

void location::rotate(vec3 rot) {
	std::move(rot.array, rot.array + 3, orientation);
	rotation = graphics::createEulerRotation(orientation);
}

void location::move(std::initializer_list<float> pos) {
	if (pos.size() == 3) {
		std::move(pos.begin(), pos.end(), position);
		translation = graphics::createTranslation(position);
	}
}

void location::move(vec3 pos) {
	std::move(pos.array, pos.array + 3, position);
	translation = graphics::createEulerRotation(position);
}

void location::resize(float factor) {
	size = factor;
	scale = graphics::createScale(factor);
}
```

```
mat4 location::getTransformation() {
	return translation * rotation * scale;	//returns the product of the 3 transformations stored by the location class
}

scene::scene() {	//create the window
	if (!glfwInit()) {	//exit if window creator couldnt be initialised
		exit(1);
	}

	//pass window arguments
	glfwSetErrorCallback(error_callback);
	glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 4);
	glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 2);
	glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

	window = glfwCreateWindow(800, 800, "Test Window", NULL, NULL);	//create the window
	if (!window) {	//exit if a window couldnt be created
		glfwTerminate();
		exit(1);
	}

	glfwMakeContextCurrent(window);
```

```
	if (!gladLoadGL()) {  //exit if functions couldnt be loaded
		glfwTerminate();
		exit(1);
	}


	glDepthFunc(GL_LESS);	//makes sure things obscured from the camera
arent renderde
	glEnable(GL_DEPTH_TEST);
}
```

3.7 shaders

3.7.1 vertexShader.vert

```
#version 420 core
layout (location = 0) in vec3 inPos;
layout (location = 1) in vec2 inTex;
layout (location = 2) in vec3 inNor;

//to be sent to the fragment shader
out vec2 UV;
out vec3 position;
out vec3 normal;

//outside inputs
uniform mat4 trans;
uniform mat4 cam;
uniform mat4 proj;

void main()
{
   gl_Position = proj * cam * trans * vec4(inPos, 1.0);    //tells opengl to put the vertex in the correctly transformed placed
   UV = vec2(inTex.x, 1.0 - inTex.y);  //corrects texture as obj has a different format to opengl
```

```
    position = vec3(trans * vec4(inPos, 1.0));  //send actual position of vertex to
frag shader

    normal = mat3(transpose(inverse(trans))) * inNor;   //send corrected normal
to frag shader

}
```

3.7.2 fragmentShader.frag

```
#version 420 core
in vec2 UV;
in vec3 position;
in vec3 normal;

out vec3 FragColor; //sends the colour of the fragment to opengl

//variables from the program
uniform sampler2D image;
uniform int lights = 1;
uniform vec3 lightPositions;
uniform float lightIntensities;
uniform vec3 cameraPosition;
uniform float reflectivity = 1.0f;
uniform float shinyness = 32.0f;
uniform float ambient;

void main()
{
    vec3 cameraDirection = cameraPosition - position;

    vec3 intensity = reflectivity * ambient * vec3(1.0f, 1.0f, 1.0f);   //calculate inital
ambient light intensity

    for (int i = 0; i < lights; ++i) {  //iterate through all lights
```

```
        vec3 lightDirection = lightPositions[i] - position;
        float lightDistance = length(lightDirection);

        //calculate diffuse intensity
        float costheta = max( dot( normalize(normal), normalize(lightDirection) ),
0.0f );
        float diffuse = reflectivity * lightIntensities * costheta;

        //calculate specular intensity
        float cosphi = max( dot( normalize(lightDirection),
normalize(cameraDirection) ), 0.0f );
        float specular = reflectivity * lightIntensities * pow(cosphi, shinyness);

        //calculate total intensity
        float lightsIntensity = (diffuse + specular) * (10 / pow(lightDistance, 2));
        intensity = intensity + (lightsIntensity * vec3(1.0f, 1.0f, 1.0f));
    }

    FragColor = intensity * texture(image, UV).rgb; //put texture and light onto
fragment
}
```

4 Testing

This video and its timestamps will be used as proof that these tests were successful

https://www.youtube.com/watch?v=2spnNFgfsDs

4.1 Feature testing

This section aims to ensure that the project meets the objectives given. Each subsection has a time stamp and an explanation to which objectives it meets and why it meets it.

4.1.1 License checking

0:00 – 1:00

1. The server-side application can be seen generating pseudo-random license keys.
2. The server-side application responds correctly to the types of valid and invalid licenses.
3. The licenses stored in the database are accessed and used to validate licenses.
4. The user-side does not let the user through if their license is invalid and does if it is.
5. Prompts are shown for failure and success on the user-side.

4.1.2 Rendering

1:00 – 4:17

1. Meshes being rendered shows that the program can use and switch between program shaders.
2. Different meshes and textures can be used.
3. Models are shown to be moving around the scene, in addition to the camera.
4. The lighting model provides dark and bright spots where they should be. The light intensity and the lights position is shown to be able to be changed in real time.
5. These meshes vertices are constructed from the OBJ files, and then rendered by the program.

4.2 Functionality testing

Some pieces of functionality still need to be testing even if they are not part of the objectives.

1:00 – 4:17

The meshes search function lets us draw the correct mesh without having to use its class identifier, meaning it must work as intended.

The scene class stores the information required by the program, letting us access import data (such as the camera and the lights).

The hashtable works as expected as the file loading class functions.

5 Evaluation

5.1 Completeness

Objective number	Section	Complete
1	Application	Y
2	Application	Y
3	Application	Y
4	Application	Y
5	Application	Y
6	Application	Y
1	Verification	Y
2	Verification	Y
3	Verification	Y
1	Extension	N
2	Extension	N
3	Extension	N

<u>Client feedback</u>

My client said he was pleased by the end product, saying it did everything it was meant to do. However, he also said that it would have been nice if there were debugging features, and if the keyCreator script had a GUI rather than being command line.

5.2 Improvements

I have managed to accomplish the objectives I set out from the start. There are still improvements that could be made though, with the program itself and my work ethic.

For example, if I had managed my time better, some of the extension objectives could've been completed.

In addition, multithreading and a scheduler program could significantly increase the performance of the program. This would likely require reconstruction of the program from the ground up, as multithreading is a very complex task to do properly, requiring locking variables and programming in a specific way to keep the program 'thread safe'.

Another improvement I could make is use a more verbose API. This would also require complete reconstruction of the program, as the previous API calls used would be completely useless. An API I could switch to is Vulkan. This API is much more complicated, requiring a lot more set-up than OpenGL but it gives the programmer much more control and much better performance.